

UKRAINIAN CATHOLIC UNIVERSITY

BACHELOR THESIS

Regularization methods in PINNs for Nonlinear Differential Equations

Author:

Davyd ILNYTSKYI

Supervisors:

Prof. Raul JIMENEZ

Prof. Oleksii IGNATENKO



*A thesis submitted in fulfillment of the requirements
for the degree of Bachelor of Science*

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences



Lviv 2026

“Job’s not finished. Job finished? I don’t think so.”

Kobe Bryant

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

Regularization methods in PINNs for Nonlinear Differential Equations

by Davyd ILNYTSKYI

Abstract

Over the past few decades, machine learning has increasingly influenced scientific computing. The field of differential equation solving has also benefited from this development, particularly through the introduction of Physics-Informed Neural Networks (PINNs). This area continues to develop rapidly, with numerous new methods introduced each year. However, the diversity of benchmarking practices makes comparisons across the literature challenging, since problem formulations can vary substantially.

This work develops a PINN evaluation framework that enables fully controlled and reproducible experiments. Using this framework, differentiation-based regularization methods, including gradient-enhanced PINNs (gPINNs), Unimodular Regularization (UR), and Jacobian Regularization (JR), are systematically compared with simpler L_1 and L_2 regularization techniques. After identifying the optimal regularization coefficients, the selected configurations were evaluated across five independent runs. The aggregated results indicate the following pattern: UR and JR achieved the best performance for the Burgers equation, gPINN for the Fisher-KPP equation, and L_1 regularization for the Allen-Cahn equation. Regularized PINNs outperformed the vanilla PINN across all considered benchmarks, demonstrating improved solution behavior in regions characterized by complex dynamics.

To the best of our knowledge, this work presents the first application and systematic evaluation of JR in the field of PINNs. In addition, it investigates the use of L_1 and L_2 regularization for the considered objective, providing further insight into the applicability of these methods. These results suggest that no single regularization technique is universally optimal, since the best-performing method depends on the considered benchmark equation. Source code can be found in [3].

Acknowledgements

I would like to express my deepest gratitude to my family, whose support made this journey possible.

I am sincerely grateful to my supervisors, Oleksii Ignatenko and Raul Jimenez, for their guidance throughout this work. Their help made this research process significantly more structured and meaningful.

I would also like to thank Pablo Tejerina and Pedro Tarancón-Álvarez for their dedicated time, insightful feedback, and helpful suggestions.

Finally, I would like to express my profound gratitude to the Armed Forces of Ukraine for defending our country and giving me the opportunity to pursue my education in Ukraine.

Contents

Abstract	ii
Acknowledgements	iii
1 Introduction	1
1.1 Motivation	1
1.2 Contributions	1
1.3 Structure Of The Thesis	2
2 Related Work	3
2.1 Physics-Informed Neural Networks	3
2.2 Failure Modes of PINNs	3
2.3 Benchmarking and Comparison Methodology	4
2.3.1 Benchmarking	4
2.3.2 Comparison Methodology	4
2.3.3 Comparison Papers	5
2.4 Domain Review	5
2.4.1 State-of-the-art Models	5
2.4.2 Sampling Techniques	6
2.4.3 Loss Reweighting Techniques	6
2.4.4 Feature Embedding	6
2.5 Regularization Functions	6
2.5.1 Gradient-Enhanced Physics-Informed Neural Networks	7
2.5.2 Unimodular Regularization	7
2.5.3 Jacobian Regularization	7
3 Theoretical Background	8
3.1 Physics-Informed Neural Networks	8
3.2 Soft and Hard constraints for PINNs	9
3.3 Finite Element Method	10
3.4 Benchmark Equations	11
3.4.1 Allen-Cahn	11
3.4.2 Burgers' Equation	12
3.4.3 Fisher-KPP	13
3.5 Regularization Functions	14
3.5.1 Jacobian Regularization	15
3.5.2 Unimodular Regularization	15
3.5.3 Gradient-Enhanced Physics-Informed Neural Networks	16
3.5.4 L1 and L2 Regularizations	16
3.6 Evaluation Metrics	17

4	Proposed Solution	18
4.1	Problem Formulation	18
4.2	Model Architecture	19
4.3	Important Decisions	19
4.4	Data Sampling	21
4.5	Results analysis	22
4.6	Limitations	22
4.7	Chapter conclusion	23
5	Experiments and Results	24
5.1	Regularization Coefficients Search	24
5.1.1	Comprehensive Results Analysis	24
5.2	Statistical Comparison of Regularization Methods	27
5.2.1	Relative L_2 Errors Across Benchmark Equations	27
5.2.2	Training Curve Analysis Across Benchmark Equations	29
5.3	Solution Analysis and Baseline Comparison	31
5.3.1	Solution Comparison	31
5.3.2	Heatmaps Analysis	31
5.3.3	Comparison with vanilla PINN	38
5.4	Chapter Summary	39
6	Conclusions	40
6.1	Results Summary	40
6.2	Limitations	41
6.3	Future Work	41
A	Appendix	42
A.1	Definitions	42
A.2	Nonlinear Differential Equations	42
A.3	Initial and boundary conditions	43
A.4	Exponential Moving Average	44
A.5	Hyperparameters configuration	45
B	Appendix	46
B.1	Computational Infrastructure	46
	Bibliography	47

List of Figures

3.1	FEM solution of the Allen-Cahn equation at different times T	11
3.2	FEM solution of the Allen-Cahn equation over the full time domain $t \in [0, 1]$	12
3.3	FEM solution of the Burgers' equation at different times T	12
3.4	FEM solution of the Burgers' equation over the full time domain $t \in [0, 1]$	13
3.5	FEM solution of the Fisher-KPP equation at different times T	14
3.6	FEM solution of the Fisher-KPP equation over the full time domain $t \in [0, 1]$	14
4.1	Optimizers comparison: Adam vs L-BFGS for Allen-Cahn benchmark equation.	20
4.2	Comparison of Fisher-KPP equation solutions for different regulariza- tion call periods.	21
5.1	Averaged training curves over 5 independent runs for Allen-Cahn, Fisher- KPP, and Burgers' equations under EMA, with $\alpha = 0.01$	29
5.2	One-dimensional solution comparison for the Allen-Cahn equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM	32
5.3	One-dimensional solution comparison for the Burgers' equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM	33
5.4	One-dimensional solution comparison for the Fisher-KPP equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM	34
5.5	Heatmap comparison of the solutions for the Allen-Cahn equation: left—neural network solution, middle—reference FEM solution, right—absolute error. Color bar denotes $u(x, t)$	35
5.6	Heatmap comparison of the solutions for the Burgers' equation: left—neural network solution, middle—reference FEM solution, right—absolute er- ror. Color bar denotes $u(x, t)$	36
5.7	Heatmap comparison of the solutions for the Fisher-KPP equation: left—neural network solution, middle—reference FEM solution, right—absolute error. Color bar denotes $u(x, t)$	37
5.8	Comparison of FEM reference, regularized(L_1) PINNs, and vanilla mod- els for the Allen-Cahn equation at $t = 0.5$	38
5.9	Comparison of FEM reference, regularized(Jacobian Regularization) PINNs, and vanilla models for the Burgers' equation at $t = 0.5$	38
5.10	Comparison of FEM reference, regularized(Unimodular Regularization) PINNs, and vanilla models for the Fisher-KPP equation at $t = 0.5$	38
A.1	Example of Exponential Moving Average method applied to Fisher- KPP equation training data.	44

List of Tables

5.1	Regularization terms which yield the best performance according to the relative L_2 error metric. Coefficients are searched for each benchmark equation.	24
5.2	Relative L^2 error for Fisher-KPP equation across regularization terms.	26
5.3	Relative L^2 error for Allen-Cahn equation across regularization terms.	26
5.4	Relative L^2 error for Burgers' equation across regularization terms.	26
5.5	Fisher-KPP equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.	27
5.6	Burgers' equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.	27
5.7	Allen-Cahn equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.	28
A.1	Hyperparameter configuration for each benchmark equation. E_1 denotes experiment 5.1, while E_2 , E_3 - experiments 5.2 and 5.3	45
B.1	HPC Cluster Hardware Specification	46
B.2	Google Colaboratory Hardware Specification	46

List of Abbreviations

PDE	P artial D ifferential E quation
DE	D ifferential E quation
IC	I nitial C ondition
BC	B oundary C ondition
PINNs	P hysics- I nformed N eural N etworks
cPINNs	C onservative P hysics- I nformed N eural N etworks
gPINN	G radient-enhanced P hysics- I nformed N eural N etwork
PIKAN	P hysics- I nformed K olmogorov- A rnold N etwork
FEM	F inite E lement M ethod
MLP	M ultilayer P erceptron
LBFGS	L imited-memory B royden- F letcher- G oldfarb- S hanno
UR	U nimodular R egularization
JR	J acobian R egularization
RAD	R esidual-based A daptive D istribution
MSE	M ean S quared E rror
L2	R elative L2 E rror
CV	C omputer V ision
NLP	N atural L anguage P rocessing
SOTA	S tate-of-the- A rt
EMA	E xponential M oving A verage

List of Symbols

u	The solution of the differential equation
$\hat{u}$	The neural network prediction of the solution
u_0	The initial condition function
g	The boundary condition function
r	The PDE residual function
x	A spatial variable
t	The time variable
z	The neural network input
N	The total number of evaluation points
N_r	The number of residual collocation points
N_{ic}	The number of initial condition collocation points
N_{bc}	The number of boundary condition collocation points
θ	The neural network parameters
λ	The regularization coefficient
ε	The Allen-Cahn equation parameter
ν	The Burgers' equation parameter
r	The Fisher-KPP equation parameter
Φ	The embedding of the input and latent representation into augmented space
H	The latent representation produced by the basis layer
K	The FEM stiffness matrix
U	The vector of unknown FEM coefficients
F	The FEM load vector
V	The space of admissible test functions
Ω	The computational domain

Dedicated to my family

Chapter 1

Introduction

1.1 Motivation

With the rapid development of artificial intelligence and computational resources, many previously intractable problems have become feasible to solve. Scientific computing, in particular, has experienced tremendous progress over the past decades [5, 16, 13]. One of the fundamental problems in this domain concerns obtaining solutions to differential equations, which are widely employed to model complex physical and biological phenomena.

Traditionally, numerical methods have been the standard approach for solving differential equations. However, over the past decade, a new paradigm has emerged in the form of Physics-Informed Neural Networks (PINNs) [16], which leverage neural networks to approximate solutions while incorporating physical laws directly into the training process.

This field is rapidly evolving, with numerous architectures and methodological improvements proposed each year [13]. Among these, regularization techniques play an important role in enhancing convergence speed, solution stability, and overall accuracy. Automatic differentiation constitutes the fundamental computational machinery of PINNs. Regularization approaches that exploit this capability therefore deserve attention, particularly in assessing whether they can yield meaningful performance improvements. Despite the breadth of proposed approaches, the literature lacks a systematic comparison of regularization methods under strictly controlled conditions, where the sole varying factor is the regularization term.

The motivation of this thesis is to address this gap by developing a unified PINN-based framework for evaluating regularization strategies. The proposed approach focuses on comparing key performance metrics across multiple differential equations, providing completely reproducible results.

1.2 Contributions

This work investigates the influence of regularization techniques on the training dynamics of PINNs, benchmarking each against a standard (vanilla) PINN. Specifically, we examine how a range of regularization strategies — including Unimodular Regularization [20], Jacobian Regularization [2], L1, L2, and gradient-enhanced PINNs (gPINN) [26] — affect convergence behavior and solution accuracy. This work presents the first implementation and systematic evaluation of JR in the context of PINNs. While L1 and L2 regularization are widely used in machine learning, here they are applied to the latent basis outputs rather than the model weights. Together, these contributions provide additional scientific insight into the behavior of regularization in PINNs. Beyond this comparative analysis, the most effective regularization variant is evaluated in comparison with numerical approaches and vanilla PINNs. It

is researched whether it is possible to improve the performance of PINNs without incorporating any major architectural changes.

The results provide practical insights into the selection of appropriate regularization methods and corresponding hyperparameters (e.g., regularization coefficients) for improving PINNs behavior.

Furthermore, a modular experimental framework is developed to facilitate the definition of governing equations, initial and boundary conditions, domain configurations, regularization strategies, and model architectures. Although the framework was developed in service of the experimental methodology rather than as a standalone contribution, it enables systematic comparison across setups and is designed to be readily extended.

Finally, an extensive set of experiments is conducted, including systematic hyperparameter exploration, with all results and observations documented to support reproducibility and further analysis.

1.3 Structure Of The Thesis

This thesis is divided into the following sections:

- **Chapter 2** surveys related works to this research field, covering fundamental work of PINNs, known failure modes of PINNs, proposed methods for enhancing their performance, state-of-the-art models, and the regularization techniques selected for comparison in this study.
- **Chapter 3** presents the mathematical background of the approaches used in this work, including PINNs, the Finite Element Method, the selected regularization functions, benchmark equations, and the evaluation metrics.
- **Chapter 4** details the experiments pipeline architecture, model, explains research methodology, important decisions and limitations.
- **Chapter 5** provides all experiments results, detailed analysis and consistent comparison of the best-yielding results regularizations with the vanilla PINNs.
- **Chapter 6** presents the conclusions of the work, discusses potential applications, and outlines possible directions for future work.
- **Appendix** introduces terms and concepts that may be unfamiliar to the general reader, provides detailed experimental configurations, and describes the computational resources used for the experiments.

Chapter 2

Related Work

2.1 Physics-Informed Neural Networks

In 2019, [16] introduced a novel approach for solving ODEs and PDEs known as Physics-Informed Neural Networks (PINNs). The key idea behind this method is that many real-world problems are characterized by limited available data but a large amount of prior knowledge in the form of scientific laws, empirically validated rules and correlations. PDEs are a representative example of such problems [13]. To incorporate this prior knowledge into the learning process, authors have proposed a modified loss function that enforces the underlying physical laws by penalizing the residual of the differential equation(DE). As a result, the model can approximate solutions of DEs without requiring labeled data, which is well-suited for problems where measurements are prohibitively expensive to obtain, or no experimental data has been captured at all.

The concept of incorporating physical constraints into neural networks was explored earlier [9]. This idea was revisited and further developed in [16], where the authors leveraged modern computational tools. In particular, the approach relies on automatic differentiation and specialized machine learning libraries, enabled by recent advances in both hardware and software technologies.

In their study, authors addressed both forward and inverse problems and demonstrated effectiveness of PINNs on a range of benchmark equations, including the Allen-Cahn, Burgers', and Schrödinger equations. For these benchmark equations, the reported relative L_2 errors were 6.99×10^{-3} , 6.7×10^{-4} , and 1.97×10^{-3} , respectively.

No universally accepted threshold exists for interpreting relative L_2 error in PINNs, as its magnitude is inherently problem-dependent, varying with the governing PDE, domain scaling, and evaluation methodology. Performance is typically evaluated by comparing results against established baselines. In this context, the results of [16] represent a foundational contribution to the field, offering the first characterization of the problem.

2.2 Failure Modes of PINNs

Even though vanilla PINNs have demonstrated their effectiveness for solving PDEs in both forward and inverse settings [13], it is worth noting their limitations. The first is PINNs solution accuracy - classical numerical solvers still produce more precise and reliable results. PINNs loses due to several factors. For example, spectral bias [6], which refers to the tendency of neural networks to learn low-frequency components of a solution more than high-frequency ones. One of the reasons for that can be neglecting the higher-order consistency of the PDE, taking into account only residual loss [27]. This also leads to poor solution approximation in regions with sharp gradients.

Next problem is the loss imbalance [13], which is caused by the combination of multiple loss components with inherently different scales and gradient magnitudes. As a result, certain loss functions can dominate the optimization process, steering the model toward satisfying only a subset of constraints. This imbalance can lead to unstable training and slower convergence [13].

2.3 Benchmarking and Comparison Methodology

2.3.1 Benchmarking

Benchmarking the PINNs model means comparing the predicted solution with the ground truth for a specific chosen PDE benchmark equation[16]. One way of acquiring reference is via analytical solution. The problem is that analytical solutions for PDEs exist only for specific ICs/BCs and certain PDEs,(if they exist at all), so it is not possible to obtain them for the generic case. To obtain solutions for generic cases, numerical solvers are used and can serve as a ground truth to assess PINNs solutions. The reason for this is that the convergence properties of these numerical methods are well understood and, in practice, they typically achieve higher accuracy than PINNs [20], making them a reliable benchmark against which PINNs predictions can be evaluated. There is a wide variety of numerical methods, but in this work, the FEM [11] will serve as the reference solution.

After acquiring the reference solution, a comparison metric should be defined. In the field of PINNs, different metrics are used: Root Mean Square Error, Mean Absolute Error, and Relative L_2 Loss are among the most commonly used in literature [13]. Also, the residual loss value is treated as a metric and used to compare different methods. However, it is important to note that the relationship between the loss function and the relative error is dictated by the DE itself. While these two quantities are generally correlated, they can differ significantly in magnitude. In this work, relative L_2 error, and residual loss are considered as the main metrics.

2.3.2 Comparison Methodology

Performing a direct comparison of the different PINN enhancement techniques from the results presented in the respective original works is a challenging task. That comes from the fact that no universal rule exists for obtaining these results. The first source of inconsistency in the literature arises from the problem formulation. In particular, each author defines the PDE setup, including the domain and the associated ICs/BCs, which can vary significantly across studies. Another important thing to mention is reproducibility - many papers do not share access to source code, so comparison demands building everything from scratch, and implementation can vary from the original, which can lead to misleading results. A further concern is optimizers - in PINNs, the choice of optimization technique can have large impact on the final result [8]. Running the model with Adam [7] or with LBGFS [10] optimizers yields different results. Even repeated runs of the same experimental setup can yield different results, as neural network parameters are randomly initialized, introducing variability into the training process.

Taken together, these factors — along with additional variations across different works such as differing loss reweighting strategies, loss function formulations, and IC/BC enforcement methods — make it largely impractical to draw reliable conclusions from raw cross-paper comparisons.

2.3.3 Comparison Papers

There are papers that offer a comparison of different PINNs improvement methods. [1] proposes a comprehensive benchmark of PINNs, with numerous method implementations and open-source code. This paper presents results from a comparison of different methods, including gPINN [26] loss function. It is shown that there is no universal method that yields the best results across different benchmark equations. Among prominent methods are PINNs with the L-BFGS [10] optimizer, which have achieved one of the lowest relative L_2 errors.

A comprehensive comparison of non-adaptive and residual-based adaptive sampling methods was proposed in [24], and the source code is also available. Authors have shown that Residual-based adaptive distribution (RAD) [24] achieves the best performance among sampling and resampling methods, yielding the lowest relative L_2 error across all experiments on the Diffusion, Burgers', Allen-Cahn, and Wave equations.

2.4 Domain Review

Since the emergence of PINNs in 2019 [16], the field has experienced rapid development and numerous methodological advancements. Research efforts have focused on multiple directions, including improvements in model architectures, training point resampling strategies, the design of novel loss functions, loss reweighting techniques, and feature embedding methods [13]. As a result, a wide range of approaches to enhance PINN performance have been proposed. Comprehensive survey of these developments is provided by [13].

2.4.1 State-of-the-art Models

In the field of PINNs, it is difficult to define the state-of-the-art (SOTA) models, as no single architecture can be universally considered the optimal [1]. Their performance is highly problem-dependent and varies with the type of DE, its formulation, and the imposed constraints. But there are some architectures worth mentioning.

Conservative PINNs (cPINNs) [4] represent a prominent domain decomposition architecture specifically designed for conservation laws. Their approach deploys separate subnetworks on spatially partitioned subdomains while enforcing flux continuity at interfaces, making them particularly effective for problems governed by conservation principles. In [18], it is shown that Physic-Informed Kolmogorov-Arnold Neural Networks (PIKANs), which represent multivariate functions as compositions of one-dimensional functions, can outperform traditional PINNs in terms of convergence speed and accuracy.

Another line of research explores the application of transformer-based architectures to the solution of PDEs. In [25], the Transolver model is introduced, demonstrating superior performance in terms of convergence speed and accuracy across multiple benchmarks, particularly for problems involving complex geometries. This approach can be considered SOTA within this specific context. Similarly, PINNsFormer [28] incorporates transformer mechanisms into the PINN framework, yielding faster convergence and superior accuracy. Notably, it reduces residual losses to 3.7×10^{-5} and 3×10^{-6} on benchmark equations, representing improvements of several orders of magnitude over standard PINNs, which yield losses of 1.6×10^{-2} and 2×10^{-1} , respectively.

2.4.2 Sampling Techniques

An additional important direction of improvement concerns the sampling of training points. Conventional strategies, such as random sampling or uniform grid-based (equally spaced) sampling, are widely used due to their simplicity; however, they are often suboptimal [13]. Adaptive sampling techniques address this limitation by allocating more points to regions where the solution is more difficult to approximate. A representative example is RAD [24], with the idea that the model is trained for N iterations on an initial set of residual points, and after a new set is sampled from a PDF proportional to the PDE residual magnitude, replacing the previous set of residual points, and the process is repeated.

2.4.3 Loss Reweighting Techniques

For PINNs employing soft IC/BC constraints (see Section 3.2), the loss function is composed of several components (see Section 3.1), making the determination of appropriate weighting between them a nontrivial task. Consequently, numerous loss reweighting strategies have been proposed. For instance, [22] introduces a method that utilizes gradient-based information to balance composite loss terms, achieving accuracy improvements of up to two orders of magnitude across various problems compared to vanilla PINNs. Another approach [19] focuses on reweighting point-wise errors rather than entire loss terms. In this method, an auxiliary network estimates the importance of each training point and assigns corresponding weights to its contribution to the overall loss. This strategy has been shown to improve convergence speed and solution accuracy over other methods.

2.4.4 Feature Embedding

PINNs have incorporated feature embedding techniques inspired by other domains, such as Computer Vision (CV) and Natural Language Processing (NLP). One of the key challenges in PINNs is the so-called spectral bias [6]. This limitation hinders the accurate approximation of solutions with sharp gradients or multi-scale behavior. Standard PINNs operate on raw spatial and temporal coordinates, requiring the network to implicitly construct high-frequency representations, which can slow down training or even prevent convergence. To address this issue, sinusoidal input embeddings, such as Fourier feature mappings, can be employed. These embeddings explicitly introduce multiple frequency components into the input space. A good example of the use of this technique is [23], even though no comparison with vanilla PINNs is provided.

2.5 Regularization Functions

In this work, the primary focus is placed on loss functions that involve derivatives of the learned solution or its transformations. As the main candidates, the following methods are considered: gPINN [26], Unimodular Regularization (UR) [20], and Jacobian Regularization (JR) [2]. In addition, L1 and L2 regularizations are included as baseline approaches commonly used in machine learning. These methods differ in terms of the aspect of the model they regularize. In particular, gPINN introduces penalties on the derivatives of the PDE residual, JR constrains the sensitivity of the network outputs with respect to the inputs, and UR imposes structural constraints on the latent representation of the solution. In contrast, L1 and L2 regularizations act directly on the magnitude of the learned representations.

2.5.1 Gradient-Enhanced Physics-Informed Neural Networks

Gradient-enhanced physics-informed neural networks (gPINNs), introduced in [26], extend standard PINNs by incorporating not only the PDE residual into the loss function but also its spatial and temporal derivatives. The key idea is that if the PDE residual is zero, its derivatives should also vanish, and enforcing this condition provides additional constraints during training. This leads to improved approximation quality by promoting higher-order consistency of the solution and supplying richer gradient information for optimization. As a result, gPINNs typically achieve better convergence to the true solution in both forward and inverse problems, often reducing the relative L_2 error by one to two orders of magnitude compared to standard PINNs [26].

2.5.2 Unimodular Regularization

Unimodular Regularization [20] constrains the latent space by penalizing its volume, measured via the determinant of the induced metric tensor. The motivation behind this approach is that the physically correct solution corresponds to a local minimum in the high-dimensional parameter space of the neural network. However, it can be difficult to reach this minimum due to the presence of narrow regions in the loss landscape that guide the optimization trajectory toward it. It is hypothesized that UR regularizes the latent space by widening regions with favorable optimization properties, thereby improving stability during gradient-based training—an effect that is especially pronounced in transfer learning.

This hypothesis is supported by the empirical findings of [20]. Throughout this work, transfer learning denotes the procedure of first training a model on a family of equations whose governing parameter lies in the range $[a, b]$, and subsequently fine-tuning the model to accommodate stiffer configurations in which the parameter extends to the range $[b, c]$. The fine-tuning to the new head for finding solutions in the stiffer regimes is much less costly than training the full model from scratch. In transfer learning experiments conducted on stiff regimes of the van der Pol oscillator, models incorporating UR attain a mean relative L_2 error of 5%, whereas standard PINNs without regularization fail to adapt effectively, yielding a substantially higher error of 52%, showing better ability of model with UR to learn new regimes.

2.5.3 Jacobian Regularization

Jacobian regularization (JR) [2] introduces an additional loss term that penalizes the norm of the derivative of the model's output with respect to its inputs, encouraging local smoothness and robustness to input perturbations. While originally explored in CV tasks, a similar idea can be adapted to PINNs. This is the first time this regularization is applied and compared in the field of PINNs.

Chapter 3

Theoretical Background

3.1 Physics-Informed Neural Networks

The loss function consists of 3 parts: residual loss, also known as physics loss, initial condition loss and boundary condition loss [16]. To obtain residual loss, the governing equation (A.2) can be rewritten in the form of a residual function:

$$r(x, t) := F \left(x, t, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial t}, \dots \right). \quad (3.1)$$

which can be used to formulate the residual loss:

$$\mathcal{L}_{\text{residual}} = \frac{1}{N_r} \sum_{i=1}^{N_r} |r(x_i, t_i)|^2. \quad (3.2)$$

Here, N_r denotes the number of collocation points for residual constraints. The 2D spatio-temporal grid is flattened into a one-dimensional array, such that the pairs (x_i, t_i) span the entire defined set of collocation points. The residual loss quantifies how well the neural network solution satisfies the DE by measuring the deviation of the PDE residual from zero at the collocation points. Collocation points are a finite set of discrete locations in the spatial-temporal domain where the neural network is enforced to satisfy the governing DE. Each loss component is evaluated over its own set of collocation points.

The initial condition loss measures the difference between the predicted solution $u(x, t_0)$ and the prescribed function $h(x, t_0)$ at the initial time $t = t_0$.

$$\mathcal{L}_{\text{initial}} = \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} |u(x_i, t_0) - g(x_i, t_0)|^2. \quad (3.3)$$

Here, N_{ic} denotes the number of collocation points for initial constraints. The boundary condition loss measures the difference between the predicted solution and the prescribed boundary function g over the set of boundary collocation points.

$$\mathcal{L}_{\text{boundary}} = \frac{1}{N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} |u(x_i, t_i) - g(x_i, t_i)|^2. \quad (3.4)$$

Here, N_{bc} denotes the number of collocation points for boundary constraints. x_i denotes points on the boundaries of the spatial domain.

Finally, the total loss can be defined as

$$\mathcal{L} = \lambda_{\text{ic}} \mathcal{L}_{\text{initial}} + \lambda_{\text{bc}} \mathcal{L}_{\text{boundary}} + \lambda_{\text{r}} \mathcal{L}_{\text{residual}}. \quad (3.5)$$

where λ_{ic} , λ_{bc} , and λ_r are coefficients that control the relative contribution of the IC, BC, residual losses, respectively.

3.2 Soft and Hard constraints for PINNs

There are two principal approaches to enforcing ICs and BCs within the PINN framework: the soft and the hard constraint methods.

In the soft constraint approach, the total training loss is decomposed into three components — a PDE residual loss, an IC loss, and a BC loss — which are minimized jointly. This was introduced in [16] and is explained in Section 3.1. This formulation is straightforward to implement and generalizes readily to complex PDEs.

The hard constraint approach, by contrast, enforces ICs and BCs by construction, embedding them directly into the network output. In the general case, the constructed solution takes the form

$$\tilde{u}(x, t) = A(x, t) + D(x, t)\hat{u}(x, t), \quad (3.6)$$

where the function $A(x, t)$ is designed such that $\tilde{u}(x, t)$ satisfies the prescribed ICs and BCs identically [12], $D(x, t)$ is a distance function that vanishes on the ICs/BCs domains, and $\hat{u}(x, t)$ is the unconstrained neural network output. This ensures that the correction term $D(x, t)\hat{u}(x, t)$ contributes nothing in those regions, so that $\tilde{u}(x, t)$ reduces exactly to $A(x, t)$ on the prescribed constraints.

This approach offers two notable advantages. First, since DEs can be highly sensitive to ICs/BCs, their exact enforcement is expected to improve solution accuracy. Second, by eliminating the need to penalize IC/BC violations during training, the hard constraint formulation can reduce the optimization burden on the network [15]. For these reasons, the hard constraint approach is adopted throughout this work.

3.3 Finite Element Method

The idea of FEM [11] is to convert strong form to weak form to get ability to solve the equation via finding a matrix. To understand it, consider the one-dimensional boundary value problem

$$\begin{aligned} -u''(x) &= f(x), & x \in (0, 1), \\ u(0) &= 0, & u(1) = 0. \end{aligned}$$

The strong form requires the solution u to satisfy the governing DE at every point in the domain Ω , with sufficient regularity such that all derivatives in the equation exist in the classical sense.

To obtain the weak form, we multiply the governing equation by a test function $v \in V$ and integrate over the domain Ω :

$$\int_0^1 -u''(x) v(x) dx = \int_0^1 f(x) v(x) dx.$$

Applying integration by parts gives

$$\int_0^1 u'(x) v'(x) dx = \int_0^1 f(x) v(x) dx.$$

Thus, we need to find $u \in V$ such that

$$\int_0^1 u'(x) v'(x) dx = \int_0^1 f(x) v(x) dx \quad \forall v \in V.$$

In the FEM, the interval $(0, 1)$ is divided into a finite number of small subintervals called elements. The solution is then approximated in a finite-dimensional space spanned by basis functions:

$$u_h(x) = \sum_{j=1}^N U_j \phi_j(x),$$

where $\phi_j(x)$ are piecewise linear basis functions and U_j are unknown coefficients. These basis functions are nonzero only on a small number of neighboring elements, which makes the resulting system sparse.

Substituting the approximate solution into the weak form and choosing test functions from the same basis leads to a system of algebraic equations

$$KU = F,$$

where K is the stiffness matrix, U is the vector of unknown coefficients, and F is the load vector.

For linear equations, FEM produces a linear system solved by standard linear algebra methods. For nonlinear equations, FEM leads to a nonlinear system, which is typically solved iteratively.

3.4 Benchmark Equations

Each of the benchmark equations includes a key parameter that controls the qualitative behavior of the solution, effectively tuning its complexity. For the Allen–Cahn equation, this parameter is ε , for the Fisher–KPP equation - r , for the Burgers' equation - ν .

3.4.1 Allen-Cahn

This equation describes phase separation dynamics driven by minimization of free energy, where interfaces between phases move and smooth out; it appears in materials science, e.g., evolution of domains in alloys or crystal growth. This equation is defined as

$$u_t = \varepsilon u_{xx} + 5u - 5u^3, \quad x \in [-1, 1], \quad t \in [0, 1],$$

where ε is defined as an input parameter equation. In this work, the value is fixed to 0.0001. Initial and boundary conditions are defined as:

$$u_0(x) = x^2 \cos(\pi x)$$

$$u(0, t) = u_0(0), \quad u(1, t) = u_0(1)$$

The choice of equation parameters and initial conditions follows [16]. Figure 3.1 illustrates the behavior of the Allen-Cahn solution for the defined IC/BC at time instants $[0.0, 0.2, 0.5, 0.8]$, while Figure 3.2 displays solution behavior over the full time domain.

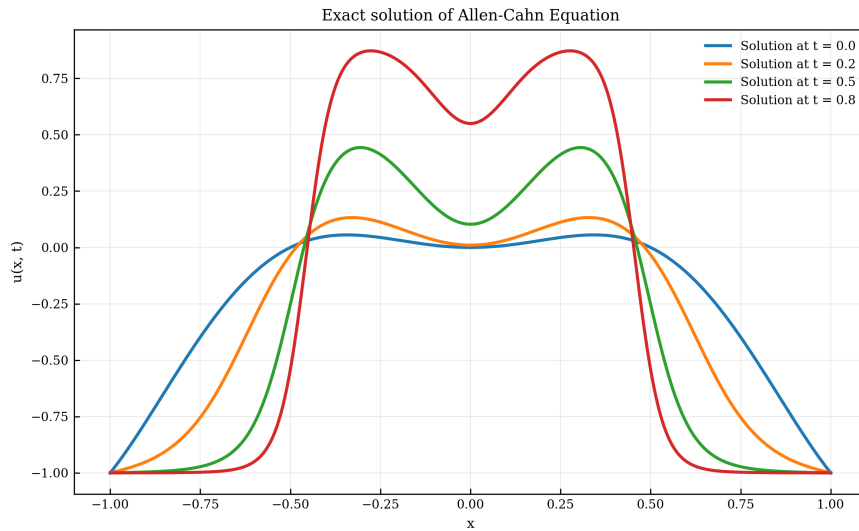


FIGURE 3.1: FEM solution of the Allen-Cahn equation at different times T .

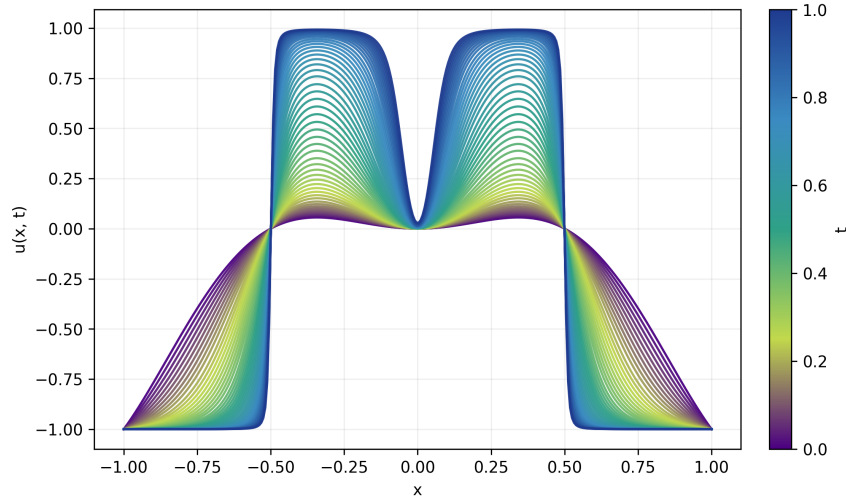


FIGURE 3.2: FEM solution of the Allen-Cahn equation over the full time domain $t \in [0, 1]$.

3.4.2 Burgers' Equation

This equation represents nonlinear advection with diffusion, capturing the formation and smoothing of shocks; it is used as a simplified model of fluid flow, traffic flow dynamics, and turbulence-like behavior.

This equation is defined as

$$u_t + uu_x = \nu u_{xx}, \quad x \in [0, 1], \quad t \in [0, 1]$$

where ν is defined as an input parameter equation. In this work, the value is fixed to $\frac{0.01}{\pi}$.

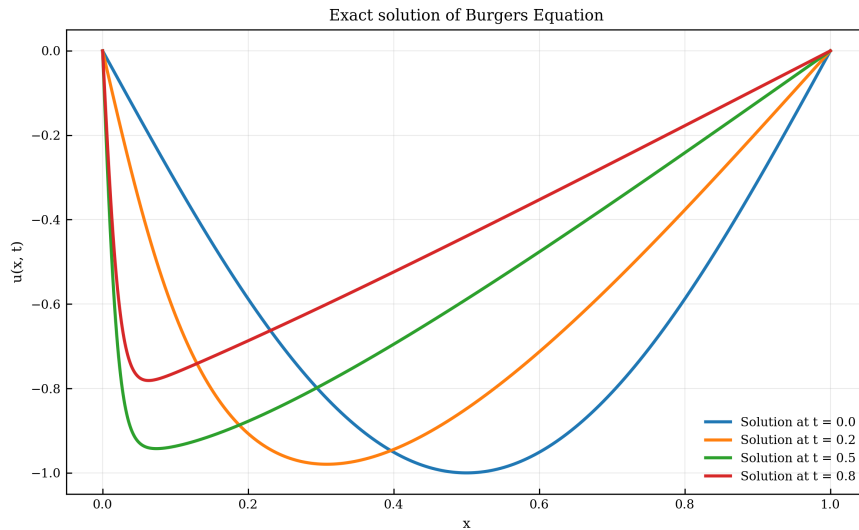


FIGURE 3.3: FEM solution of the Burgers' equation at different times T .

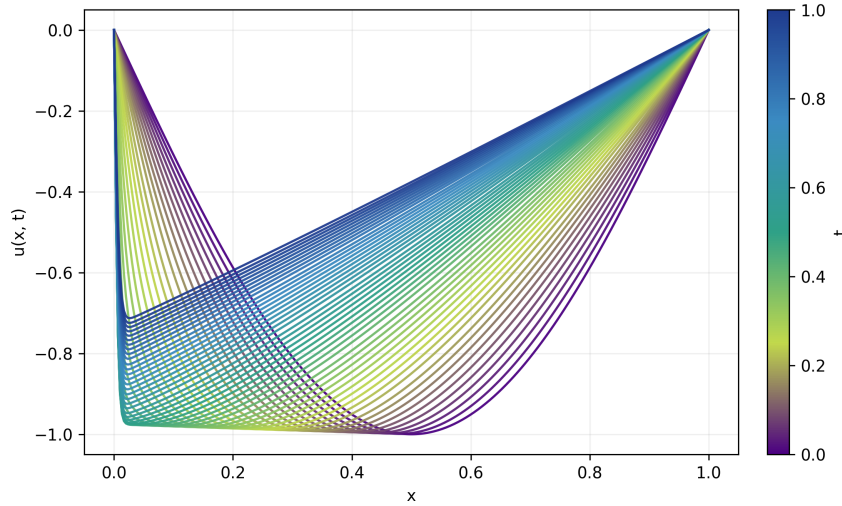


FIGURE 3.4: FEM solution of the Burgers' equation over the full time domain $t \in [0, 1]$.

Initial and boundary conditions are defined as:

$$u(x, 0) = -\sin(\pi x)$$

$$u(0, t) = 0, \quad u(1, t) = 0$$

This equation setup, including the specific choice of initial and boundary conditions, follows [16]. Figure 3.3 illustrates the behavior of the Burgers' equation solution for the defined IC/BC at time instants $[0.0, 0.2, 0.5, 0.8]$, while Figure 3.4 illustrates solution behavior over the full time domain.

3.4.3 Fisher-KPP

This equation models reaction–diffusion processes with logistic growth, capturing how a quantity spreads through space while saturating; it describes phenomena like population invasion waves or the spread of advantageous genes in biology. This equation is defined as

$$u_t = u_{xx} + ru(1 - u), \quad x \in [0, 1], \quad t \in [0, 1],$$

where r is defined as an input parameter equation. In this work, the value is fixed to 0.01. Initial and boundary conditions are defined as:

$$u(x, 0) = \frac{e^{-x} - e^{-1}}{1 - e^{-1}} \tag{3.7}$$

$$u(0, t) = 1, \quad u(1, t) = 0$$

Figure 3.5 illustrates the behavior of the Fisher-KPP equation solution for the defined IC/BC at time instants $[0.0, 0.2, 0.5, 0.8]$, while Figure 3.6 illustrates solution behavior over the full time domain.

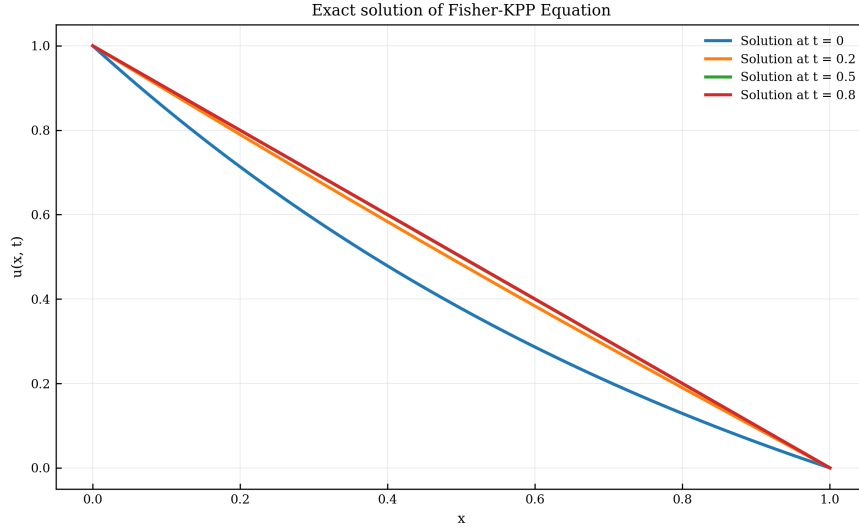


FIGURE 3.5: FEM solution of the Fisher-KPP equation at different times T .

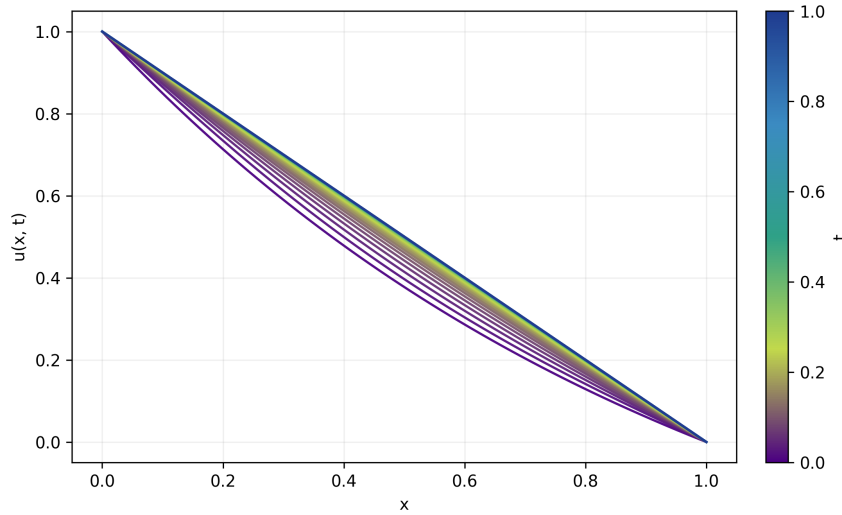


FIGURE 3.6: FEM solution of the Fisher-KPP equation over the full time domain $t \in [0, 1]$.

3.5 Regularization Functions

Regularization is a technique used to reduce overfitting and improve the generalization ability of neural networks. In the context of PINNs, regularization can speed up convergence of the model and enforce physical constraints to the system. It modifies the original optimization objective by adding a penalty term, weighted by a regularization hyperparameter λ . This can be expressed as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \mathcal{R}(\theta), \quad (3.8)$$

where $\mathcal{L}_{\text{task}}$ is the task-specific loss function — the PDE residual loss in the context of this work, $\mathcal{R}(\theta)$ is the regularization term, θ denotes the model parameters, and λ controls the strength of regularization.

3.5.1 Jacobian Regularization

Let $z \in \mathbb{R}^n$ denote the input to the neural network, and let $H(z) = (H_1(z), \dots, H_d(z)) \in \mathbb{R}^d$ denote the output of the last hidden layer (body), where n is the input dimension and d is the latent-space dimension.

Jacobian regularization is defined through the squared Frobenius norm of the Jacobian of the basis layer output with respect to the network input:

$$J_H(z) = \frac{\partial H(z)}{\partial z} \in \mathbb{R}^{d \times n},$$

$$\mathcal{L}_{\text{JR}} = \|J_H(z)\|_F^2 = \sum_{i=1}^d \sum_{j=1}^n \left(\frac{\partial H_i(z)}{\partial z_j} \right)^2.$$

Jacobian regularization penalizes the sensitivity of the model output with respect to its input. A small input perturbation changes the output approximately as a linear term involving the Jacobian. If the Jacobian is large, even tiny input changes cause large output shifts. By minimizing the Jacobian norm, the model becomes smooth and stable to perturbations. This way, instead of only fitting the data, the model is forced to behave consistently in a neighborhood around each point — which is exactly what you want in PINNs and PDE settings, where small input variations should not produce chaotic outputs.

3.5.2 Unimodular Regularization

The latent space is embedded into the augmented space through

$$\Phi(z) = (z_1, \dots, z_n, H_1(z), \dots, H_d(z)) \in \mathbb{R}^{n+d}.$$

The full Jacobian of this embedding with respect to the latent vector z is

$$\frac{\partial \Phi}{\partial z} = \begin{pmatrix} 1 & 0 & \cdots & 0 & \frac{\partial H_1}{\partial z_1} & \frac{\partial H_2}{\partial z_1} & \cdots & \frac{\partial H_d}{\partial z_1} \\ 0 & 1 & \cdots & 0 & \frac{\partial H_1}{\partial z_2} & \frac{\partial H_2}{\partial z_2} & \cdots & \frac{\partial H_d}{\partial z_2} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & \frac{\partial H_1}{\partial z_n} & \frac{\partial H_2}{\partial z_n} & \cdots & \frac{\partial H_d}{\partial z_n} \end{pmatrix} \in \mathbb{R}^{n \times (n+d)}.$$

The induced metric on the latent-space hypersurface is then defined by

$$(g_{\mu\nu})_{n \times n} = \frac{\partial \Phi}{\partial z} \left(\frac{\partial \Phi}{\partial z} \right)^T.$$

Equivalently, each element of the metric tensor can be written as

$$g_{\mu\nu}(z) = \delta_{\mu\nu} + \sum_{i=1}^d \frac{\partial H_i}{\partial z_\mu} \frac{\partial H_i}{\partial z_\nu},$$

Therefore,

$$g_{\mu\mu}(z) = 1 + \sum_{i=1}^d \left(\frac{\partial H_i}{\partial z_\mu} \right)^2, \quad g_{\mu\nu}(z) = \sum_{i=1}^d \frac{\partial H_i}{\partial z_\mu} \frac{\partial H_i}{\partial z_\nu}, \quad \mu \neq \nu.$$

Using the notation

$$a_{\mu i} = \frac{\partial H_i}{\partial z_\mu},$$

the metric can be written as

$$(g_{\mu\nu})_{n \times n} = \begin{pmatrix} 1 + \sum_{i=1}^d a_{1i}^2 & \sum_{i=1}^d a_{1i}a_{2i} & \cdots & \sum_{i=1}^d a_{1i}a_{ni} \\ \sum_{i=1}^d a_{2i}a_{1i} & 1 + \sum_{i=1}^d a_{2i}^2 & \cdots & \sum_{i=1}^d a_{2i}a_{ni} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^d a_{ni}a_{1i} & \sum_{i=1}^d a_{ni}a_{2i} & \cdots & 1 + \sum_{i=1}^d a_{ni}^2 \end{pmatrix}.$$

Finally, the scalar quantity regularized in UR is

$$g(z) = \det[g_{\mu\nu}(z)],$$

and the loss term is written as

$$\mathcal{R}_{\text{UR}} = (\sqrt{g(z)} - 1)^2.$$

The idea behind this regularization is explain in Section 2.5.2.

3.5.3 Gradient-Enhanced Physics-Informed Neural Networks

gPINN loss minimizes the derivatives of the PDE residual with respect to the input variables.

Let the residual be $r(z)$. Then gPINN is written as:

$$\mathcal{R}_{\text{gPINN}} = \sum_{k=1}^n \left\| \frac{\partial r}{\partial z_k} \right\|^2$$

where z_k denotes the k -th input variable.

Gradient-enhanced PINNs extend the standard PINN framework by incorporating not only the PDE residual $r(x, t)$ into the loss function, but also its spatial and temporal derivatives $\nabla r(x, t)$. The idea behind is, if the governing PDE is satisfied, then both the residual and its derivatives should vanish. Standard PINNs enforce only $r(x, t) = 0$, losing additional information contained in the gradients of the residual. By enforcing $\nabla r(x, t) = 0$, gPINNs enforce stronger physical constraints on the solution, requiring the model to match both the value and local behavior of the PDE. This leads to improved approximation, particularly in regions with steep gradients.

3.5.4 L1 and L2 Regularizations

L1 and L2 regularization are closely related norm-based regularization methods. In particular, L1 regularization is based on the ℓ_1 norm,

$$\mathcal{R}_{L1} = \|H\|_1 = \sum_{i=1}^d |H_i|, \quad (3.9)$$

while L2 regularization is based on the squared ℓ_2 norm,

$$\mathcal{R}_{L2} = \|H\|_2^2 = \sum_{i=1}^d H_i^2. \quad (3.10)$$

Here, $H \in \mathbb{R}^d$ denotes the latent representation produced by the last layer of the body network.

L1 regularization encourages sparsity in the latent representation and may lead some of its components exactly to zero, whereas L2 regularization penalizes large values more smoothly and therefore encourages smaller, more uniformly distributed latent components.

3.6 Evaluation Metrics

The main evaluation metrics used in this study are residual loss, relative L_2 error, and pointwise absolute error. Relative L_2 Error is defined as

$$L_2^{\text{rel}} = \frac{\|u - \hat{u}\|_2}{\|u\|_2} = \frac{\sqrt{\sum_{i=1}^N (u_i - \hat{u}_i)^2}}{\sqrt{\sum_{i=1}^N u_i^2}} \quad (3.11)$$

where N is the total number of evaluation points, u_i is the reference solution at the i -th point obtained via FEM, $\hat{u}_i$ is the corresponding prediction of the PINN. Pointwise absolute error is defined as

$$E_{\text{abs}}(x, t) = |u(x, t) - \hat{u}(x, t)| \quad (3.12)$$

where $u(x, t)$ denotes the reference solution evaluated at the point (x, t) , while $\hat{u}(x, t)$ represents the corresponding prediction of the PINN at the same point.

Chapter 4

Proposed Solution

The main goal of this work is to investigate how different regularization techniques affect the training process and behavior of PINNs. It is researched whether it is possible to improve the performance of PINNs without incorporating any major architectural changes.

A fair comparison of regularization methods requires all experimental conditions to be identical, except for the regularization term. Conducting such experiments requires a flexible pipeline capable of varying key parameters. Developing this pipeline was therefore the primary initial objective of this work, using the codebase from [21] as a starting point. Throughout the research, extensive configuration capabilities were developed, encompassing: PDE/ICs/BCs definitions, reference solution method selection (numerical or analytical, where available), domain and data-sampling settings, random seed, multi-head IC configuration, model parameters, as well as config artifact saving and visualization. The result is a configurable framework in which all of the above parameters can be systematically adjusted and controlled.

This chapter is structured as follows: firstly, the problem is formulated. After, chosen model architecture is introduced. Next, key design decisions are discussed, including the choice of optimizer, the regularization methods call period, and coefficients search, among other aspects. The chapter then describes the data sampling approach used to collect results, collocation points, along with a justification of its validity. This is followed by an explanation of the results analysis methods and a justification of the selected benchmark equations. The chapter concludes with a discussion of the work's limitations and summary.

4.1 Problem Formulation

For a given PDE, the solution is generally not unique; instead, there exists a set of admissible functions that satisfy the equation. To obtain a specific solution, additional constraints in the form of initial and boundary conditions are imposed.

PDEs are often solved through the use of numerical solvers. These produce discrete solutions evaluated on a specified grid, and for specified BCs or/and ICs. If a new solution for a different discretization is required, the whole code must be re-run, which is one of the limitations of these numerical methods. A novel approach for this problem is to make use of PINNs [16], which produce continuous approximation of the solution, which can be evaluated at arbitrary points in the spatial-temporal domain.

In the field of PINNs, numerous approaches have been proposed to improve the training process. This work investigates how different regularization techniques which are based on computing derivatives impact training process in case of convergence behavior, speed and prediction accuracy. The research is conducted for a set of problems, these are 1D forward PDEs: the Fisher-KPP, Burgers' and Allen-Cahn equations.

4.2 Model Architecture

The standard multilayer perceptron (MLP) was chosen as the main architecture for this work, as it is widely used in the PINNs literature [13]. As the activation function, $\tanh(x)$ was selected. It was shown in [14] that this function yields decent results for PINNs.

The neural network input is a 3-dimensional vector

$$\mathbf{z} = (x, t, r),$$

where x is the spatial coordinate, t is the temporal coordinate, and r is a parameter appearing in the DE.

The network employed in this study consists of input, body, basis, and head layers. This architecture was proposed in [20] as a suitable one for next setup: the body layer learns the solution components for a family of PDEs and encodes them in a latent basis representation. The head is designed to construct a solution corresponding to a specific initial condition, while the overall model employs multiple heads in parallel, each responsible for a distinct initial condition within the prescribed set. In that way, the model is encouraged to learn a solution not for one specific PDE and IC/BC, but for a family of PDEs and specific IC/BC. The baseline neural network architecture was adapted from [21] and further modified for the purposes of this study.

A reliable comparison of regularization methods is ensured by fixing the initial model state and using predefined random seeds for each experiment run. This approach eliminates inconsistencies arising from different initial model states, thereby ensuring reliable conclusions and full reproducibility of the experiments.

4.3 Important Decisions

Initially, Adam [7] was chosen as the primary optimizer for this work. But, for a number of experiments, it has shown poor performance on training for more complex equations such as Allen-Cahn and Burgers' equations, while yielding good results for simpler one - the Fisher-KPP equation. This behavior was observed independently of the size of the network, the number of epochs, the regularization functions, and other hyperparameters. For this sake, it was decided to add another optimizer widely used in the domain - L-BFGS [10]. This is a second order optimizer, which means that it has to compute 2nd derivatives to do the optimization. For this reason, Adam is much cheaper in terms of computational cost, although it gives generally worse results 4.1. L-BFGS has improved convergence for the Burgers' equation and decreased training time from hours to dozens of minutes. As a final design choice, both optimizers are employed, as it is commonly considered good practice [17] to combine Adam and L-BFGS, using Adam during the initial N training epochs and L-BFGS during the final K epochs.

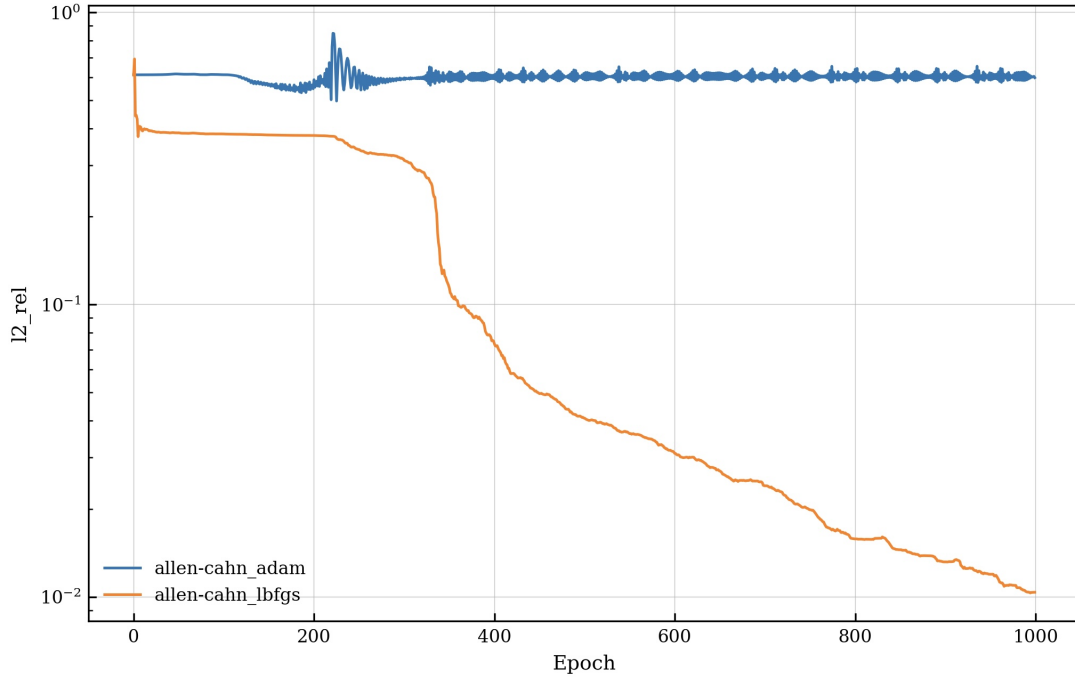
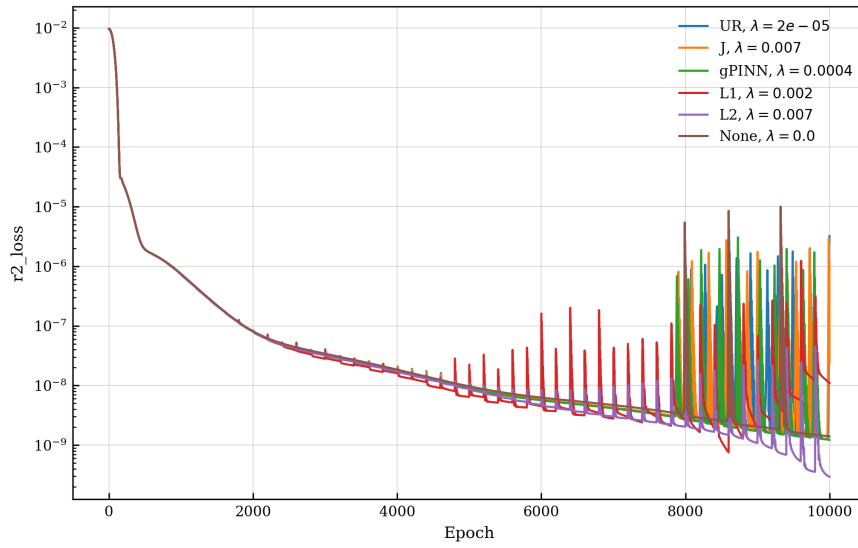


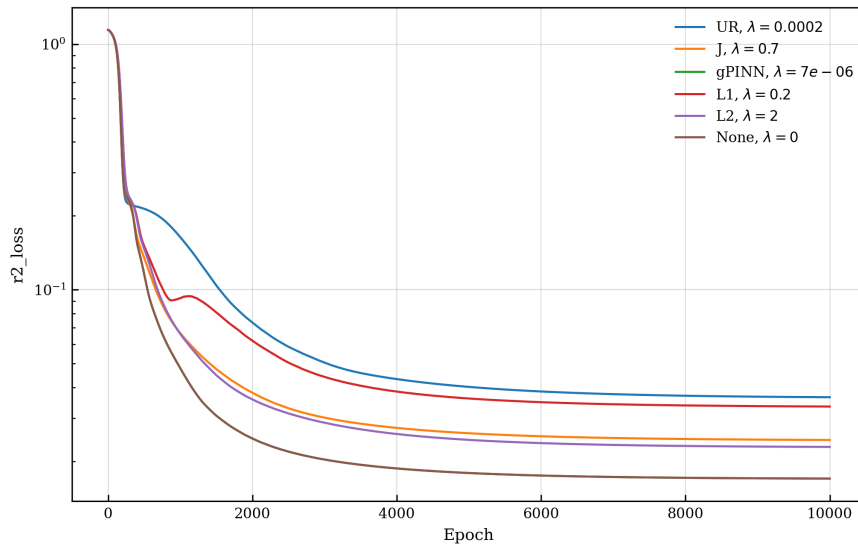
FIGURE 4.1: Optimizers comparison: Adam vs L-BFGS for Allen-Cahn benchmark equation.

The sampling period of the regularization functions was also adjusted. In [20], it is stated that the regularization update interval should be sufficiently large, since computing the UR regularization is costly. Frequent updates may cause the regularization term to dominate the PDE residual loss, thereby stalling or even halting the training process. Having this in mind, the first experiments were run with regularization periods values of 500, 200, and 10. The idea was to compensate rare regularization call with a big regularization term. During the experiments, regardless of the magnitude of the coefficients, the PDE residual error became indistinguishable from that of the vanilla PINN loss. This behavior is displayed in Figure 4.2. Given that, a decision was made to reduce the sample period to 1. This way, regularized models have started to diverge from vanilla PINNs loss, impacting the training process.

The next stage involves identifying the regularization terms that yield the best performance within each regularization method. For the same regularizations those terms can differ between benchmark equations. For this purpose, instead of searching regularization coefficients for the most difficult benchmark equation and after applying those to the others, each equation was run with a rich scope of regularization terms.



(A) Period = 200



(B) Period = 1

FIGURE 4.2: Comparison of Fisher-KPP equation solutions for different regularization call periods.

4.4 Data Sampling

Data is collected at each training epoch. Specifically, the following quantities are recorded: the value of the PDE residual loss, and the relative L_2 error. This enables visualization and subsequent analysis of the learning dynamics of the solution throughout the training process. Collocation points are sampled on a uniform grid to maintain simplicity.

While the residual loss can be directly evaluated during training, the computation of relative L_2 error requires a reference solution, as explained in Section 3.3.

The predicted solution of the model is compared with the reference solution at a fixed time $T = 0.5$, with equation parameters fixed at their training values. Evaluation is performed on a spatially refined grid, exceeding the resolution used for training.

During training, the values of the PDE residual loss and other evaluation metrics often exhibit fluctuations, which can obscure underlying trends. To obtain a clearer representation of the metric behavior, the exponential moving average (EMA) is applied as a smoothing technique. Example of this method is shown in Section A.1.

4.5 Results analysis

To ensure statistically reliable results, the number of experiments must be sufficient to meet error requirements; typically, at least 30 runs are needed given Central Limit Theorem. However, running 30 experiments demands enormous computational power and time, which is not feasible under the time limitations and computational resources available for the realization of this work. Still, it is a common practice in the field of machine learning to consider results acquired across 3-5 different runs as a reliable ones. Accordingly, a final comparison is conducted using 5 runs with different random seeds using the best regularization terms discovered from previous experiments. The results are evaluated based on the average residual loss and and relative L_2 error.

Final plots display the mean of the mentioned metrics across all training runs, with the metric values on the y-axis and the epoch number on the x-axis. This makes it possible to compare real impact of different regularizations. Conducting experiments this way would make result statistically reliable and convenient.

These results identify which regularization function minimizes PDE residual loss and relative L_2 error over a statistically reliable number of runs. For each benchmark equation, all regularization methods are evaluated starting from the same initial model weights, ensuring that any differences in performance reflect the effect of regularization alone rather than random variation in initialization. Evaluating across multiple benchmark equations, rather than a single or most complex case, ensures that conclusions are not skewed by the characteristics of any one equation family.

4.6 Limitations

It is important to mention the limitations of this work. These include the architecture, computation power, PINNs and other limitations.

One of the initial ideas of this work was to compare how those regularizations impact the process of transfer learning for different equations. For this reason [21] architecture was chosen as a baseline. However, due to the high computational cost of this setup — increasing the number of heads from 1 to 2 required over an order of magnitude more training time — the focus was shifted to investigating the impact of regularization in a simpler case: the same multi-head architecture, but used as a single-head with fixed ICs/BCs and equation parameter.

The hyperparameters governing the L-BFGS optimizer — specifically the depth and the number of epochs — were determined through empirical tuning, with the selected values corresponding to those that minimized both the PDE residual loss and the relative L_2 error. Similarly, the size of the neural network was chosen empirically, balancing two competing objectives: minimizing training time and achieving the lowest possible relative L_2 error.

A further limitation concerns the choice of benchmark equations. Fisher-KPP was selected for its low training cost, enabling efficient hyperparameter tuning, while Burgers' equation and Allen-Cahn are well-established benchmarks in the field [16]. Nevertheless, several important problem classes remain unexplored, including chaotic PDEs, high-dimensional PDEs, and inverse problems. These present substantially

greater computational demands and, in the case of inverse problems, would require significant architectural modifications beyond the scope of this work.

In the UR [20] paper, the regularization call period was set to a larger value, as a small period was noted to cause UR to dominate the loss, preventing the model from effective learning. It is possible that this behavior is present here, but the aim is to mitigate it through a small regularization term.

Another limitation of the approach is solution accuracy. Although PINNs have shown significant improvements as a method [13], numerical solvers still achieve higher accuracy.

4.7 Chapter conclusion

This chapter presents a framework for investigating the effect of regularization techniques on the training dynamics and performance of PINNs. The core objective is to enable a fair and reproducible comparison by keeping all experimental conditions fixed while varying only the regularization term. To achieve this, a highly configurable experimental pipeline was developed, supporting flexible definitions of PDEs, ICs, BCs, domain, random seeds, model configurations and other.

Careful control of randomness through fixed seeds ensures reproducibility and isolates the effect of regularization. Key design decisions include the combined use of Adam and L-BFGS optimizers improve model prediction accuracy.

The problem is formulated in the context of non-unique PDE solutions, where uniqueness is enforced through definition of ICs and BCs. This work focuses on three benchmark equations: Fisher–KPP, Burgers’, and Allen–Cahn. These are used to assess how derivative-based regularization methods influence convergence speed, stability, and solution accuracy.

The data collection and evaluation pipeline is explained: residual loss and relative L_2 error are captured during training. Results are analyzed across multiple runs to improve statistical reliability. Comparisons are made using averaged metrics to isolate consistent patterns. This work emphasizes evaluating regularization methods across multiple equations, instead of one complex benchmark equation. This is done to avoid biased conclusions.

Finally, the chapter outlines key limitations, including computational constraints, restricted exploration of complex PDE classes, and reliance on empirically tuned hyperparameters. While PINNs provide flexibility and continuous solutions, their accuracy still lags behind traditional numerical solvers. Overall, the chapter establishes a controlled experimental foundation for analyzing the role of regularization in PINNs.

Chapter 5

Experiments and Results

5.1 Regularization Coefficients Search

To identify optimal regularization terms, a series of experiments was conducted to find such terms that minimize the relative L_2 error most effectively. Detailed architectural specifications for each benchmark equation are presented in Table A.1. The primary distinction between this and the subsequent experiments lies in the number of L-BFGS optimization epochs. In the present experiment, 150 epochs are employed, whereas in the following experiments this number is increased to 300.

For each of the three benchmark equations, a total of 51 experimental runs were conducted, resulting in 153 runs overall. This experimental design arises from evaluating five regularization methods, each tested across ten distinct values of the regularization coefficient, in addition to a vanilla PINN configuration without regularization. The regularization coefficient search is performed over the range $[10^{-7}, 10^2]$. The regularization term values which yield the best results are listed in Table 5.1.

The results demonstrate that the optimal value of the regularization coefficient is not universal, but instead depends on the specific PDE under consideration. This observation justifies the adopted experimental protocol, where hyperparameter selection is performed independently for each benchmark equation. Another observation is that, for some benchmark equations, the regularization values that produce the lowest metric value lie on the boundaries of the sampled region. For example, this is the case for gPINN regularization across all three equations. It is possible that values outside the sampled range could yield better results; however, in this research, we treat the best-performing sampled values as the optimal ones.

TABLE 5.1: Regularization terms which yield the best performance according to the relative L_2 error metric. Coefficients are searched for each benchmark equation.

Equation	λ_{L_1}	λ_{L_2}	λ_J	λ_{UR}	λ_{gPINN}
Burgers'	10^{-4}	10^2	10^{-1}	10^{-6}	10^{-7}
Fisher-KPP	10^{-6}	10^{-6}	10^{-4}	10^{-6}	10^{-7}
Allen-Cahn	10^{-5}	10^{-5}	10^{-7}	10^{-7}	10^{-7}

5.1.1 Comprehensive Results Analysis

Table 5.2 presents the relative L_2 errors for the Fisher-KPP equation across different regularization methods and coefficient values. The results indicate that the explored range of regularization coefficients is generally sufficient for this benchmark, as the majority of optimal values lie within the considered interval. An exception is observed

for gPINN regularization, where the best performance is attained at the lower boundary of the search range, suggesting that smaller coefficient values may yield further improvements.

Overall, the Fisher-KPP equation demonstrates low approximation errors, with the best result achieving 4.8×10^{-6} using gPINN regularization with $\lambda = 10^{-7}$. This outcome is consistent with the relatively simple nature of the benchmark equation. Furthermore, the results indicate that regularized models outperform the vanilla PINN, which yields a relative error of 2.5×10^{-5} .

Table 5.3 reports the relative L_2 errors for the Allen-Cahn equation across different regularization methods and coefficient values. In contrast to the Fisher-KPP case, three regularization strategies attain their optimal performance at the boundary of the explored coefficient range, indicating that smaller values may yield further improvements.

The observed error magnitudes are moderate, on the order of $\mathcal{O}(10^{-2})$. Among the configurations evaluated, the lowest relative error of 8.8×10^{-3} is attained by the L_1 -regularized model with regularization parameter $\lambda = 10^{-5}$. For this benchmark equation, which follows the Fisher-KPP behavior, usage of regularization yields a marked improvement in predictive accuracy: the vanilla PINN baseline produces a substantially higher relative error of 4.9×10^{-2} , underscoring the benefit of regularization for this class of problems.

Table 5.4 reports the relative L_2 errors for the Burgers' equation across different regularization methods and coefficient values. The results indicate that the sampled range of regularization coefficients is generally adequate, as most optimal values lie within the explored boundaries. However, in contrast to the previous two benchmark equations, the present optimal boundaries terms are at both extremes of the coefficient range, - at the smallest and largest tested values. This suggests that potential improvements are achievable outside the current search interval.

Overall, the error magnitudes for this benchmark are the largest observed across all tested equations, ranging from 10^{-1} to 10^{-2} . This is expected, as Burgers' equation is the most challenging among the selected benchmark problems. JR yields the lowest relative L_2 error with $\lambda = 10^{-1}$. This benchmark equation also shows better relative L_2 error with regularized model, as vanilla PINN produces relative error of 1.3×10^{-1} .

The results consistently demonstrate that regularization techniques reliably enhance the performance of vanilla PINNs, with all three benchmark equations yielding lower relative L_2 errors compared to the unregularized baseline. Among the methods evaluated, JR, gPINN, and L_1 regularization achieved the most favorable error metrics. Notably, the strong performance of L_1 regularization is somewhat unexpected: as the simplest method in the comparison — one that introduces no differential modifications to the underlying physics — it nevertheless attains the best results on the Allen-Cahn equation, suggesting that even lightweight regularization strategies can be surprisingly effective.

TABLE 5.2: Relative L^2 error for Fisher-KPP equation across regularization terms.

Coefficient	L1	L2	JR	UR	gPINN
10^{-7}	1.2×10^{-5}	3.6×10^{-5}	10×10^{-5}	9.5×10^{-5}	4.8×10^{-6}
10^{-6}	9.7×10^{-6}	6×10^{-6}	7×10^{-5}	1.8×10^{-5}	3.3×10^{-5}
10^{-5}	2.7×10^{-5}	5.5×10^{-5}	8.8×10^{-5}	5.6×10^{-5}	3×10^{-5}
10^{-4}	6×10^{-5}	5.9×10^{-5}	1.5×10^{-4}	7.7×10^{-5}	4.2×10^{-5}
10^{-3}	1×10^{-4}	9.4×10^{-5}	1.1×10^{-4}	6.7×10^{-5}	2.4×10^{-5}
10^{-2}	1.7×10^{-4}	1.1×10^{-4}	7.2×10^{-5}	3.7×10^{-4}	3.5×10^{-5}
10^{-1}	1.4×10^{-4}	8.4×10^{-5}	7.4×10^{-5}	1.4×10^{-4}	1.2×10^{-5}
10^0	9.7×10^{-3}	1.3×10^{-4}	7.2×10^{-5}	5×10^{-4}	1.5×10^{-3}
10^1	9.7×10^{-3}	1.6×10^{-4}	1.2×10^{-4}	2.7×10^{-4}	6.3×10^{-2}
10^2	9.7×10^{-3}	7.6×10^{-5}	9.2×10^{-5}	2.4×10^{-3}	8.9×10^{-2}

TABLE 5.3: Relative L^2 error for Allen-Cahn equation across regularization terms.

Coefficient	L1	L2	JR	UR	gPINN
10^{-7}	1.7×10^{-2}	2.8×10^{-2}	1.5×10^{-2}	7.1×10^{-2}	9.4×10^{-2}
10^{-6}	1.2×10^{-1}	2.7×10^{-2}	3×10^{-2}	3.3×10^{-1}	3.3×10^{-1}
10^{-5}	8.8×10^{-3}	1.1×10^{-2}	1.6×10^{-2}	3.8×10^{-1}	3.9×10^{-1}
10^{-4}	1.5×10^{-2}	1.1×10^{-2}	2.1×10^{-1}	4.2×10^{-1}	3.9×10^{-1}
10^{-3}	3.7×10^{-1}	1.8×10^{-2}	4.4×10^{-2}	4.2×10^{-1}	3.9×10^{-1}
10^{-2}	3.5×10^{-1}	3.5×10^{-1}	1.8×10^{-1}	4×10^{-1}	4.2×10^{-1}
10^{-1}	2.7×10^{-1}	3.6×10^{-1}	3.4×10^{-1}	4×10^{-1}	4.7×10^{-1}
10^0	2.5×10^{-1}	3.1×10^{-1}	3.5×10^{-1}	5.1×10^{-1}	4.8×10^{-1}
10^1	2.9×10^{-1}	1×10^{-1}	4×10^{-1}	5.4×10^{-1}	5.3×10^{-1}
10^2	4.9×10^{-1}	3×10^{-1}	3.9×10^{-1}	5.3×10^{-1}	4.6×10^{-1}

TABLE 5.4: Relative L^2 error for Burgers' equation across regularization terms.

Coefficient	L1	L2	JR	UR	gPINN
10^{-7}	1×10^{-1}	1.4×10^{-1}	7.7×10^{-2}	1.2×10^{-1}	1.7×10^{-1}
10^{-6}	1.5×10^{-1}	1.4×10^{-1}	8.1×10^{-2}	5.7×10^{-2}	1.7×10^{-1}
10^{-5}	1.5×10^{-1}	1.1×10^{-1}	1.1×10^{-1}	4.2×10^{-1}	2.6×10^{-1}
10^{-4}	9.6×10^{-2}	1.3×10^{-1}	1.1×10^{-1}	3.1×10^{-1}	2.8×10^{-1}
10^{-3}	1.2×10^{-1}	1.3×10^{-1}	6.5×10^{-2}	3.8×10^{-1}	2.4×10^{-1}
10^{-2}	3.4×10^{-1}	1.2×10^{-1}	4×10^{-2}	3.7×10^{-1}	3.1×10^{-1}
10^{-1}	3×10^{-1}	1×10^{-1}	1.9×10^{-2}	4×10^{-1}	4.2×10^{-1}
10^0	3.9×10^{-1}	6.3×10^{-2}	4.1×10^{-1}	5.3×10^{-1}	4.5×10^{-1}
10^1	4×10^{-1}	2×10^{-1}	3.8×10^{-1}	5.8×10^{-1}	4.5×10^{-1}
10^2	5×10^{-1}	5.8×10^{-2}	3.8×10^{-1}	5.8×10^{-1}	4.2×10^{-1}

5.2 Statistical Comparison of Regularization Methods

The model architecture is kept the same as in the previous experiment, with the exception that the number of L-BFGS epochs is increased from 150 to 300. The detailed hyperparameter setup is reported in Table A.1.

5.2.1 Relative L_2 Errors Across Benchmark Equations

In this experiment, the regularization coefficients that achieved the lowest relative L_2 error in the previous experiment are fixed. To evaluate the behavior of the regularization methods in a more statistically reliable manner, all models within each benchmark equation are kept identical, except for the regularization method being applied. For a given random seed, the model weights are initialized identically for all models for specific benchmark equation, and the training procedure is performed under the same experimental configuration. In total, this experiment comprises 90 independent runs.

TABLE 5.5: Fisher-KPP equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.

Regularization	Relative L_2 error
UR	$6.7 \times 10^{-5} \pm 2.1 \times 10^{-5}$
JR	$3.8 \times 10^{-5} \pm 2.8 \times 10^{-5}$
gPINN	$1.2 \times 10^{-5} \pm 9.4 \times 10^{-6}$
L1	$1.4 \times 10^{-5} \pm 3.8 \times 10^{-6}$
L2	$1.7 \times 10^{-5} \pm 6.3 \times 10^{-6}$
None	$1.9 \times 10^{-5} \pm 1.3 \times 10^{-5}$

TABLE 5.6: Burgers' equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.

Regularization	Relative L_2 error
UR	$5.5 \times 10^{-2} \pm 2.0 \times 10^{-2}$
JR	$6.1 \times 10^{-2} \pm 4.2 \times 10^{-2}$
gPINN	$1.5 \times 10^{-1} \pm 1.3 \times 10^{-2}$
L1	$1.5 \times 10^{-1} \pm 1.7 \times 10^{-2}$
L2	$1.6 \times 10^{-1} \pm 6.2 \times 10^{-2}$
None	$1.4 \times 10^{-1} \pm 1.3 \times 10^{-2}$

TABLE 5.7: Allen-Cahn equation: mean and standard deviation of the relative L_2 error. Regularizations are configured with the best terms from experiment 5.1.

Regularization	Relative L_2 error
UR	$2.2 \times 10^{-2} \pm 5.7 \times 10^{-3}$
JR	$1.5 \times 10^{-2} \pm 9.2 \times 10^{-3}$
gPINN	$3.0 \times 10^{-1} \pm 8.8 \times 10^{-2}$
L_1	$6.2 \times 10^{-3} \pm 1.0 \times 10^{-3}$
L_2	$6.3 \times 10^{-3} \pm 2.3 \times 10^{-3}$
None	$1.3 \times 10^{-2} \pm 2.1 \times 10^{-3}$

For each benchmark equation, random seeds from 46 to 50 are used, resulting in five independent runs. For every run, the relative L_2 error is recorded, after which the mean and standard deviation are computed. This procedure makes it possible to assess whether a given regularization method provides consistently better performance compared with the other evaluated methods.

Table 5.5 shows mean and standard deviation across five runs for Fisher-KPP equation. Across all regularizations, gPINN shows the best results yielding mean and standard deviation value of $1.2 \times 10^{-5} \pm 9.4 \times 10^{-6}$. Second-best result shows L_1 regularization, with values of $1.4 \times 10^{-5} \pm 3.8 \times 10^{-6}$.

Overall, the error magnitudes are on the order of 10^{-5} for all regularization methods, showing small difference in relative L_2 error. For this benchmark equation UR produces the least favorable results, while yielding superior performance for Burgers' equation in the next experiment 5.6. It demonstrates that the choice of regularization must be tailored to the specific equation under consideration.

Table 5.6 shows mean and standard deviation across runs for Burgers' equation. Across all regularizations, Unimodular Regularization shows the best results yielding mean and standard deviation value of $5.5 \times 10^{-2} \pm 2.0 \times 10^{-2}$. Second-best result shows JR, with values of $6.1 \times 10^{-2} \pm 4.2 \times 10^{-2}$.

Overall, the error magnitudes for this benchmark are the largest observed across all tested equations, with the best-performing regularization methods converging to an error on the order of 10^{-2} .

Table 5.7 shows mean and standard deviation across runs for Allen-Cahn equation. Across all regularizations, L_1 shows the best results yielding mean and standard deviation value of $6.2 \times 10^{-3} \pm 1.0 \times 10^{-3}$. Second-best result shows L_2 regularization, with values of $1.7 \times 10^{-5} \pm 6.3 \times 10^{-6}$. The error magnitudes for this benchmark are on the order of 10^{-3} for the best regularization methods.

An important observation is that L_1 and L_2 regularizations rank among the top-performing methods for this benchmark equation; moreover, L_1 regularization achieves the second-best performance on the Fisher-KPP equation.

The conducted multi-run evaluation demonstrates that regularization has a consistent and measurable impact on PINN performance, but no single method is universally optimal across all benchmark equations. These findings highlight that the effectiveness of a regularization strategy is strongly problem-dependent, highlighting the need to match regularization choices to the specific properties of the underlying DE. Another observation is, while complex regularization strategies achieve the best performance, simple techniques such as L_1 and L_2 regularization also attain leading results for complex Allen-Cahn benchmark equation.

5.2.2 Training Curve Analysis Across Benchmark Equations

During the training process, residual loss and relative L_2 error were recorded at each epoch. The mean values over multiple independent runs were computed to obtain an averaged loss trajectory for each benchmark equation. The resulting trajectories are depicted in Figure 5.1.

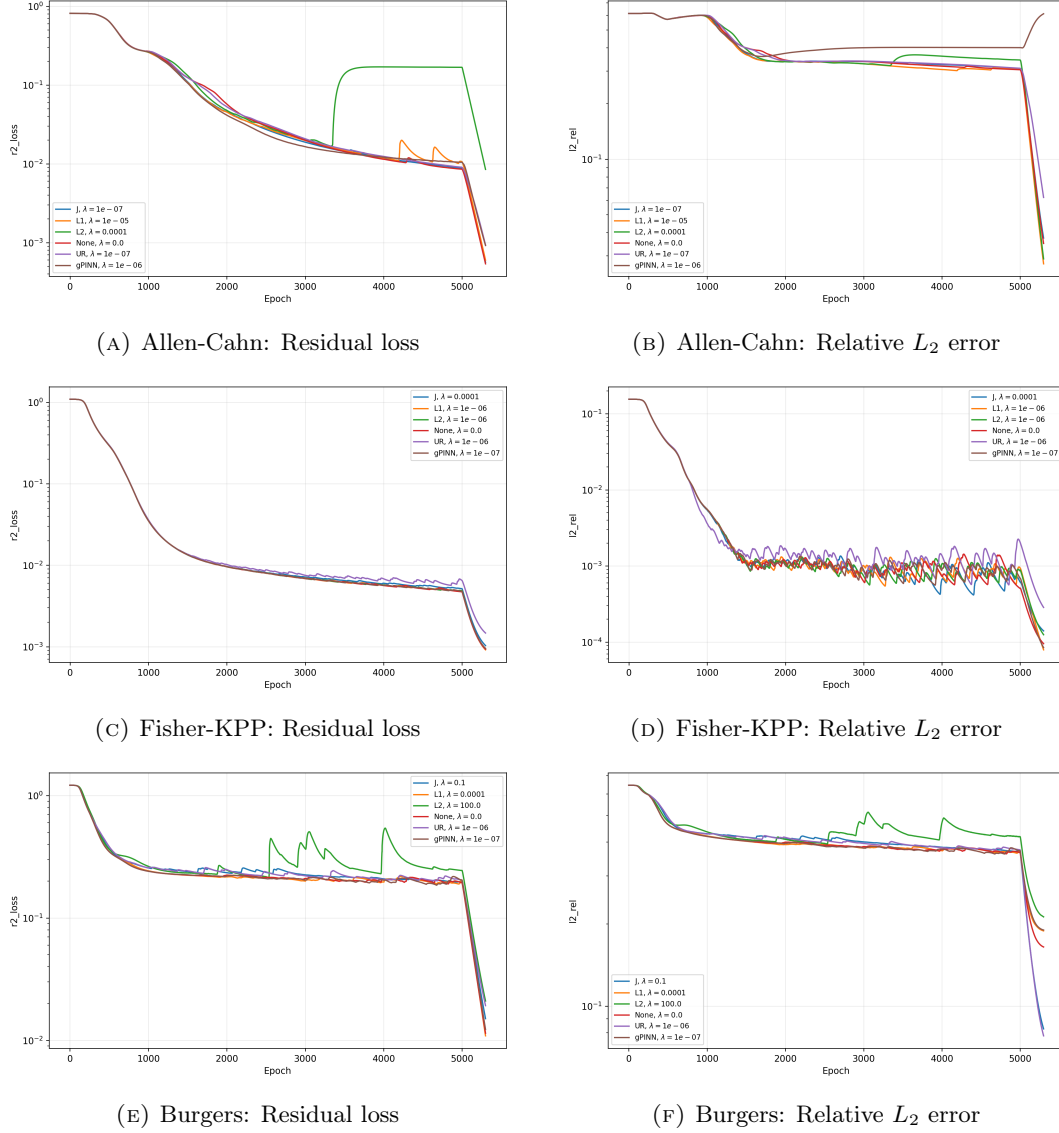


FIGURE 5.1: Averaged training curves over 5 independent runs for Allen-Cahn, Fisher-KPP, and Burgers' equations under EMA, with $\alpha = 0.01$.

A consistent pattern emerges in the regularization behavior across the benchmark equations: the metric minimization trajectories are largely shared among all regularization strategies, with only minor deviations. This is expected, as regularization terms are designed to refine model behavior incrementally rather than induce fundamental changes in the optimization dynamics.

Exception is observed for the Allen-Cahn equation, where the residual loss exhibits significant deviations under L_2 regularization. Another one is the relative L_2 error showing atypical behavior under gPINN regularization, even though corresponding residual loss continues to decrease. For L_2 regularization, the residual loss eventually

resumes a minimizing trend after an initial transient phase. It is hypothesized that gPINN regularization would display a similar pattern if trained over a sufficiently large number of epochs.

An additional phenomenon is observed for the Burgers' equation: although the residual loss trajectories appear qualitatively similar across all regularization methods, the relative L_2 error decreases substantially under UR and JR, with the trend suggesting continued improvement. In contrast, the other regularization methods appear to converge to local minima.

Finally, for the Allen-Cahn equation, the relative error initially decreases smoothly until it reaches a local minimum, after which it destabilizes when further minimization is attempted. This indicates that the error decreases steadily up to a certain point, after which the optimization process may become unstable, likely due to sensitivity in the loss landscape.

5.3 Solution Analysis and Baseline Comparison

This experiment follows the same architectural configuration as the preceding study 5.2, with the exception that only one run is conducted, and the random seed used for comparison is set to 52.

5.3.1 Solution Comparison

For each benchmark equation, plot to the right contains solution predicted by PINN at time $T = 0.5$ alongside a reference numerical solution computed via the FEM. The plot to the right displays absolute error between the reference and predicted solutions, enabling a systematic assessment of both the predicted solution behavior and its deviation from the reference.

Figure 5.2 illustrates the solution behavior for the Allen-Cahn equation. Among the evaluated approaches, L_1 regularization achieves the lowest relative error, whereas gPINN yields the highest. Overall, the model captures the solution well across most of the domain. However, the central region — where the solution exhibits high complexity — is more challenging to learn. The behavior observed in this region provides insight into how different regularization strategies influence the model’s capacity to resolve intricate solution features. In this case, it shows L_1 high ability to improve solution behavior.

Figure 5.3 illustrates the solution behavior of the Burgers’ equation across different regularization schemes. The JR achieves the lowest relative error, whereas the gPINN exhibits the highest. The main numerical difficulty is associated with the steep gradient region formed by the nonlinear advection term at the start of the domain. This localization of the shock provides a clear basis for distinguishing the relative influence of each regularization approach on solution accuracy. This high-gradient structure provides a useful basis for comparing how different regularization approaches affect solution accuracy.

Figure 5.4 illustrates the solution behavior of Fisher-KPP equation across different regularization schemes. This is the easiest benchmark equation among chosen and for time $t = 0.5$ is shows relatively simple solution plot. The UR yields the best result, while the worst is given by the vanilla PINN.

5.3.2 Heatmaps Analysis

Heatmaps provide a comprehensive visualization of the solution across the entire spatiotemporal domain. Following standard convention, the vertical axis represents time and the horizontal axis represents the spatial coordinate x , while the color encodes the value of the solution function at each point (x, t) . The corresponding color bar indicates the mapping between color and solution magnitude. Illustrations of heatmaps are presented in Figures 5.5, 5.6, and 5.7.

The absolute error heatmaps of predicted and reference solutions highlight the regions in which the model struggles to produce accurate predictions. For the Allen-Cahn equation, the most challenging regions are located at the later portion of the time domain near the center of the spatial domain. In the case of Burgers’ equation, errors are concentrated at the beginning of the spatial domain and persist across the entire temporal range. For the Fisher-KPP equation, the primary difficulty arises within the temporal interval $[0, 0.3]$, where the most significant solution dynamics occur before the solution converges to its stable form. These findings align with the earlier 1D solution plots, where top-performing regularizations also show strong results in the heatmaps.

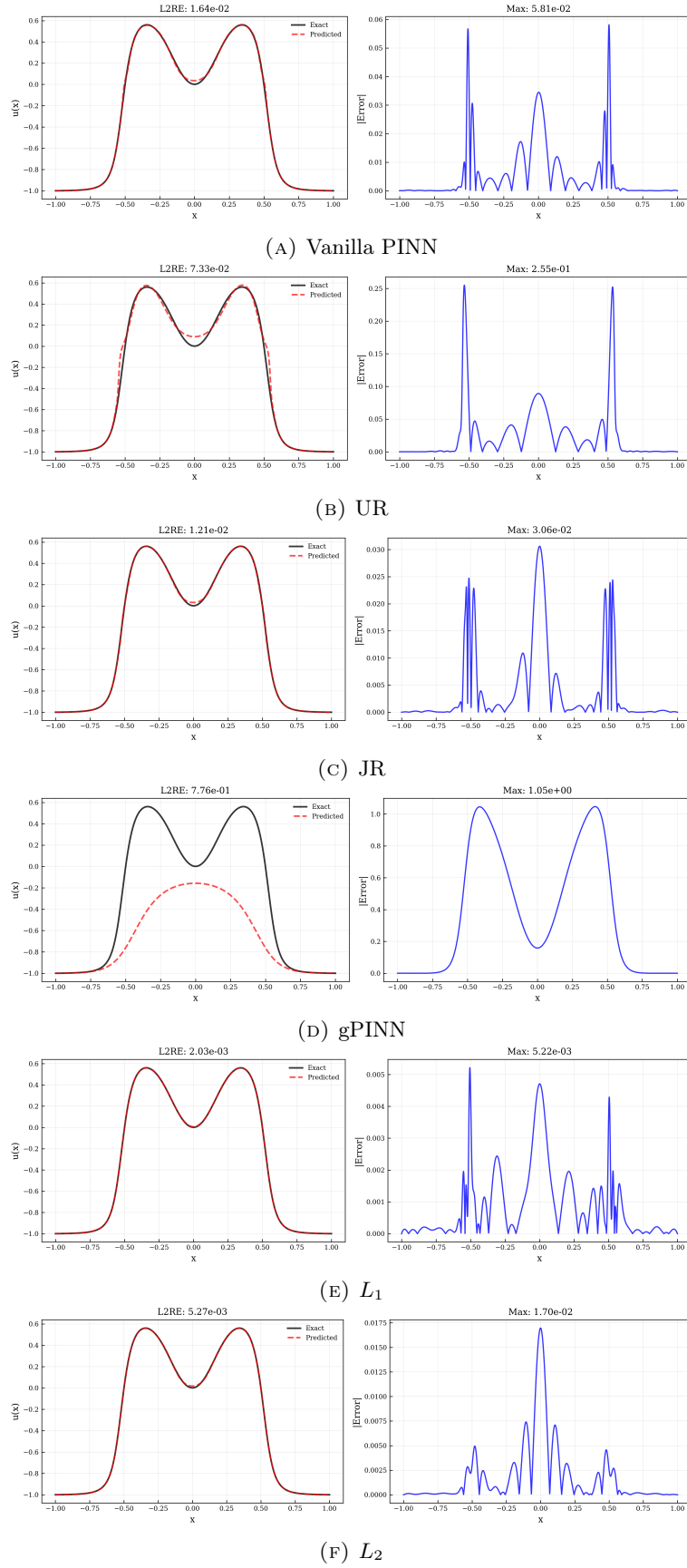


FIGURE 5.2: One-dimensional solution comparison for the Allen-Cahn equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM

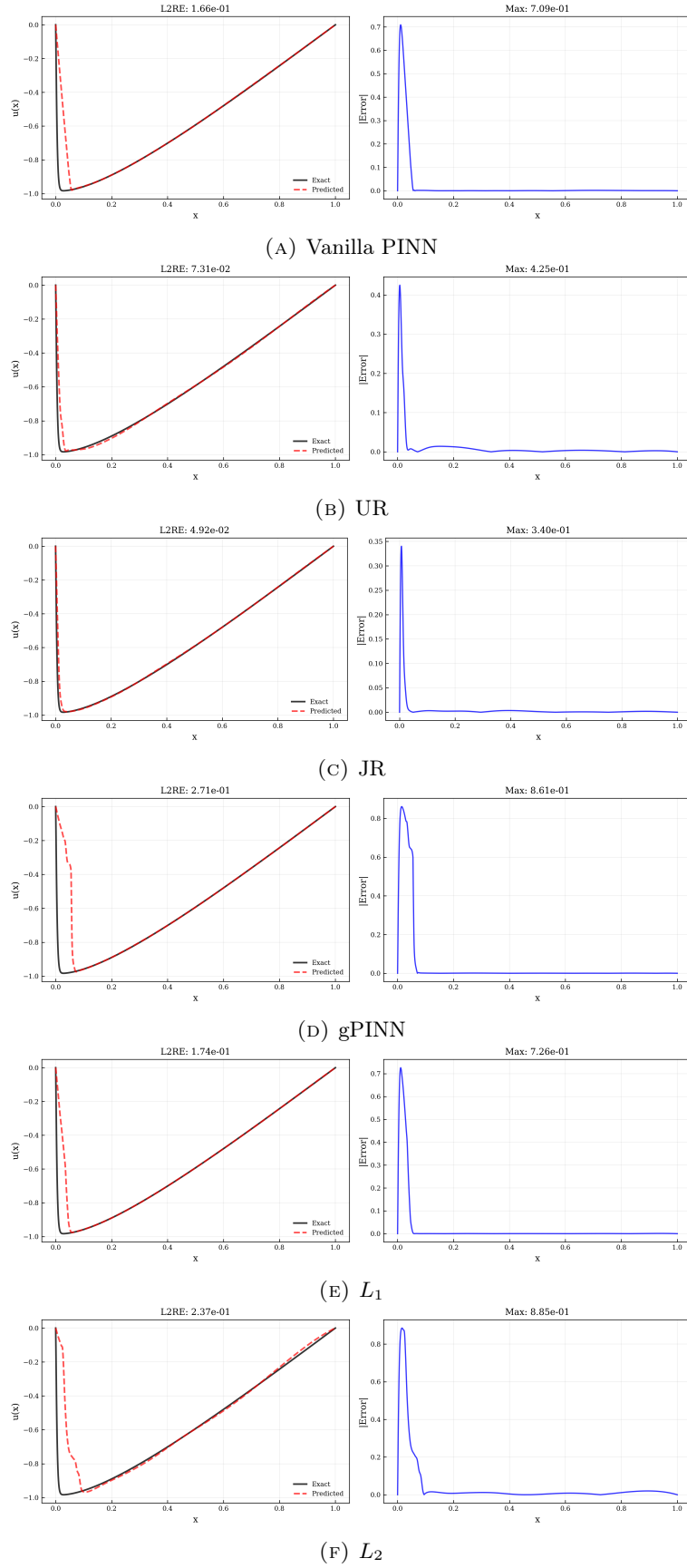


FIGURE 5.3: One-dimensional solution comparison for the Burgers' equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM

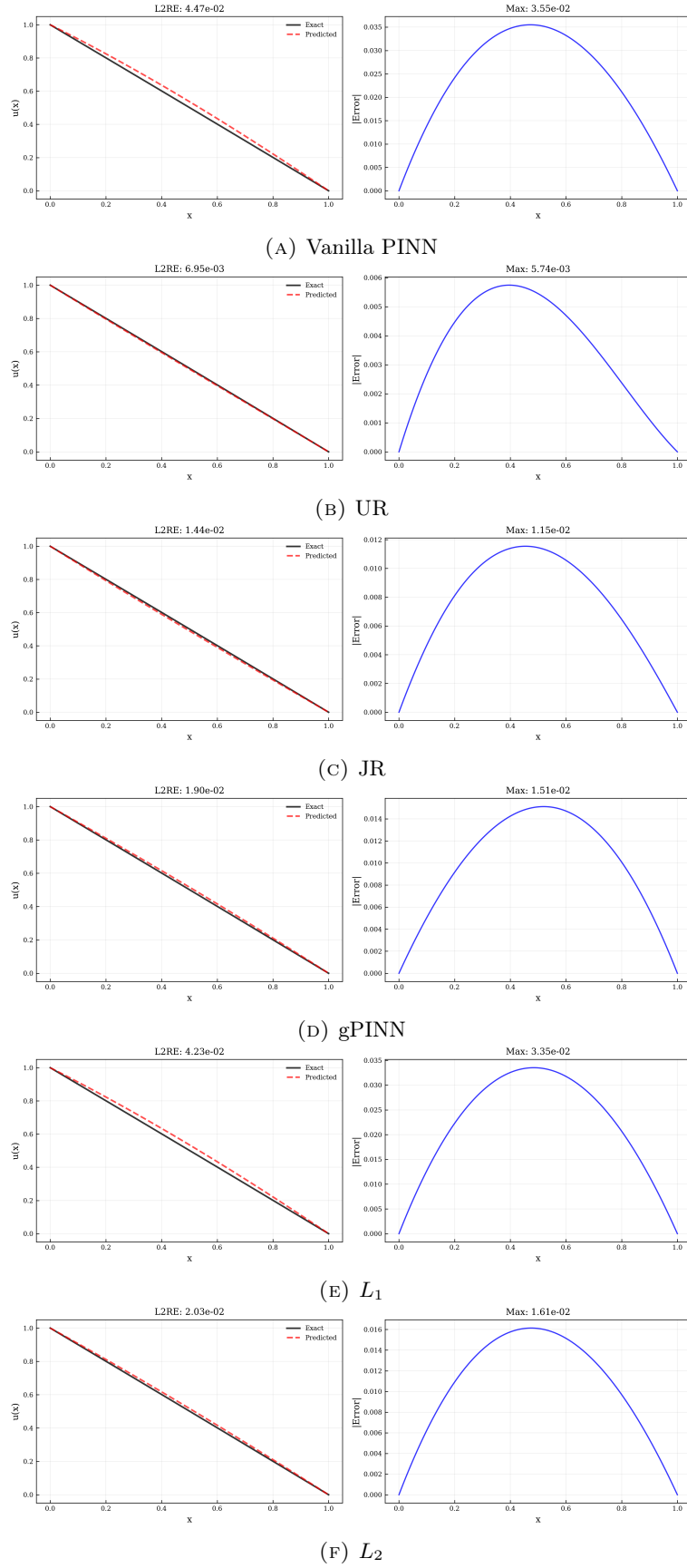
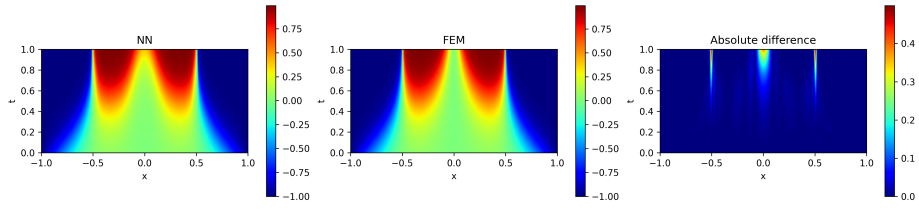
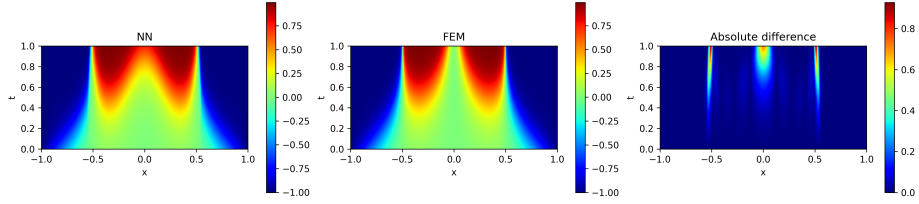


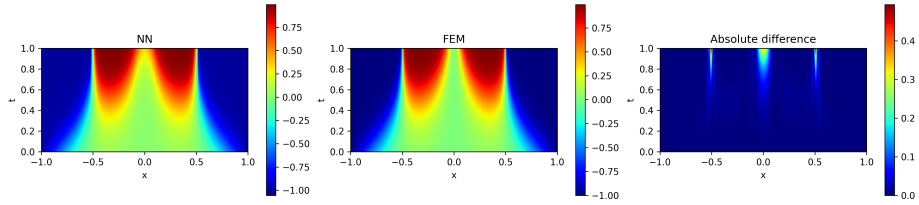
FIGURE 5.4: One-dimensional solution comparison for the Fisher-KPP equation. Left - PINNs and FEM solution, Right - Absolute Difference between PINNs and FEM



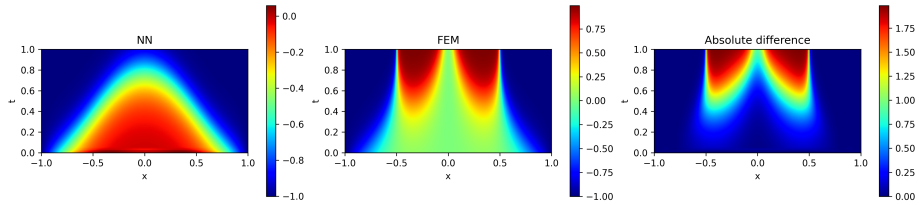
(A) Vanilla PINN



(B) UR



(C) JR



(D) gPINN

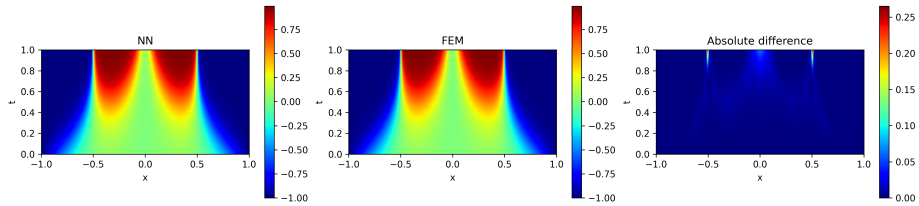
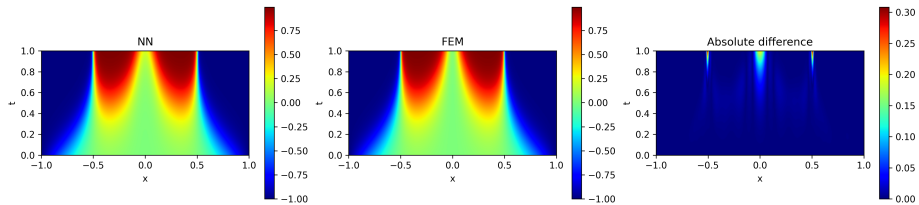
(E) L_1 (F) L_2

FIGURE 5.5: Heatmap comparison of the solutions for the Allen-Cahn equation: left—neural network solution, middle—reference FEM solution, right—absolute error. Color bar denotes $u(x, t)$.

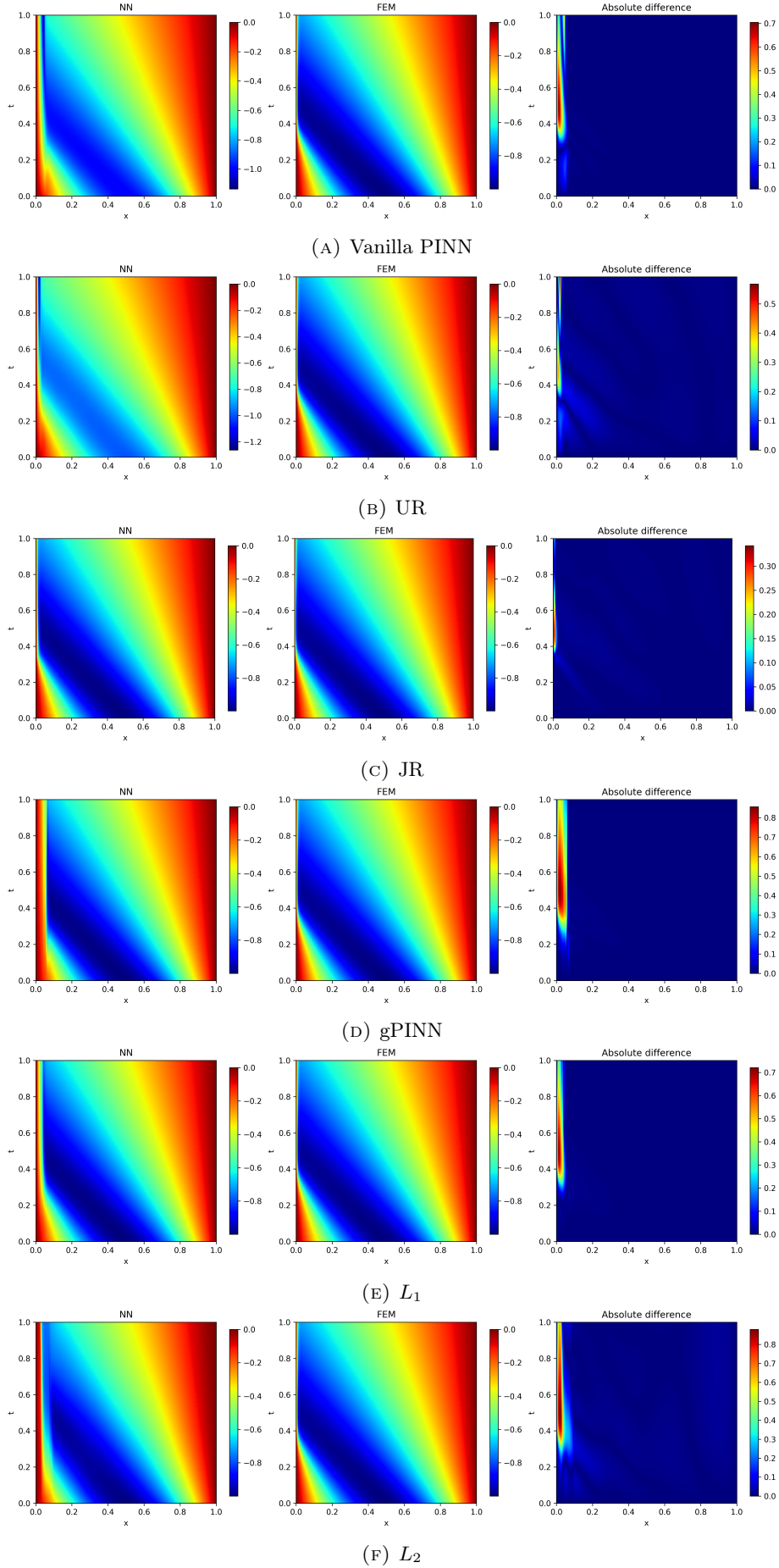


FIGURE 5.6: Heatmap comparison of the solutions for the Burgers' equation: left—neural network solution, middle—reference FEM solution, right—absolute error. Color bar denotes $u(x, t)$.

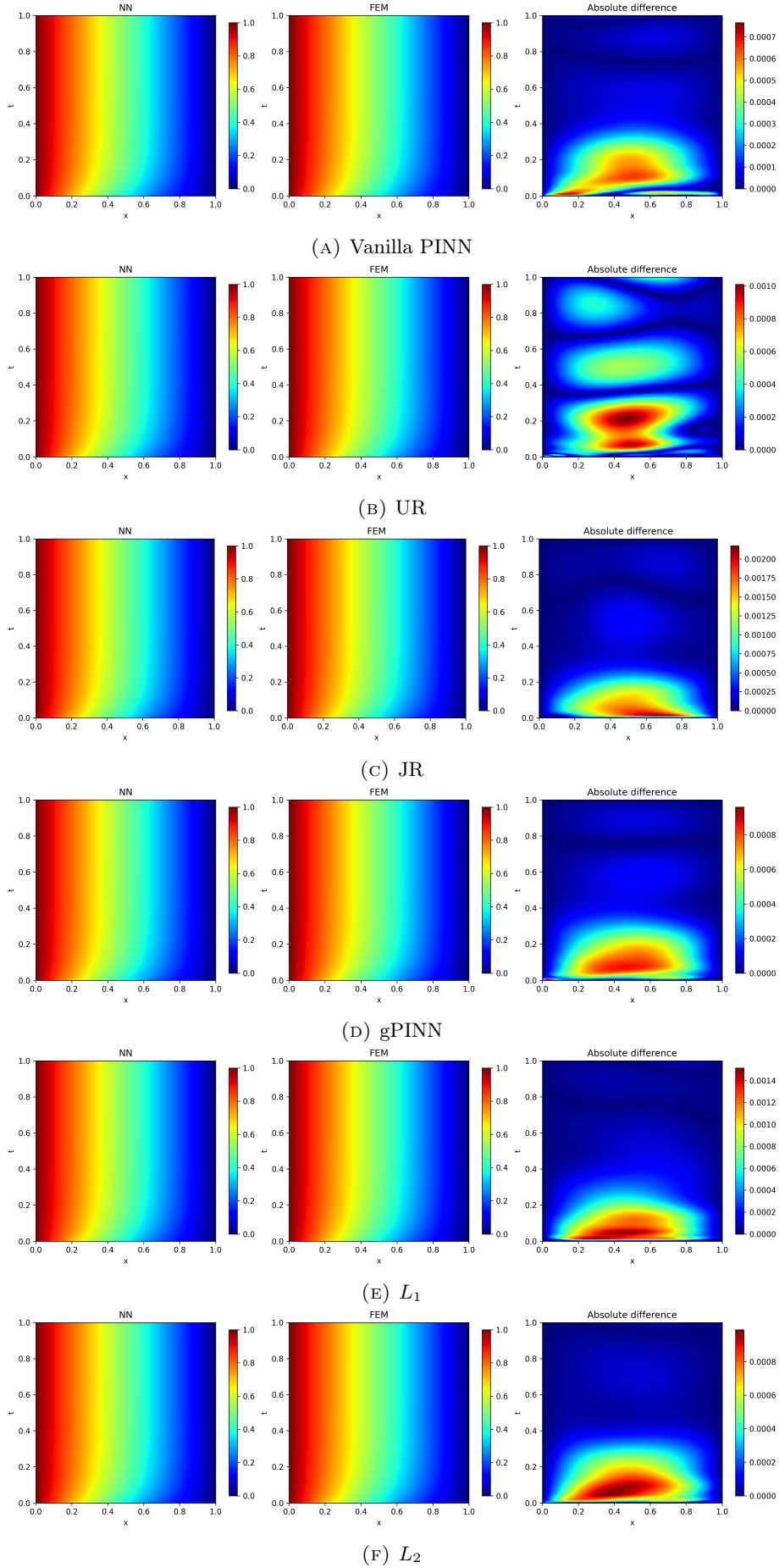


FIGURE 5.7: Heatmap comparison of the solutions for the Fisher-KPP equation: left—neural network solution, middle—reference FEM solution, right—absolute error. Color bar denotes $u(x, t)$.

5.3.3 Comparison with vanilla PINN

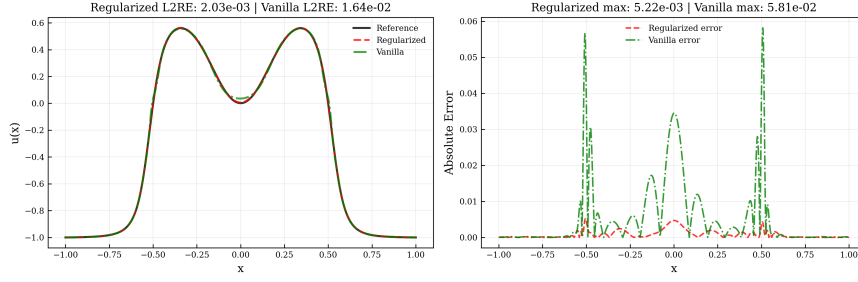


FIGURE 5.8: Comparison of FEM reference, regularized(L_1) PINNs, and vanilla models for the Allen-Cahn equation at $t = 0.5$.

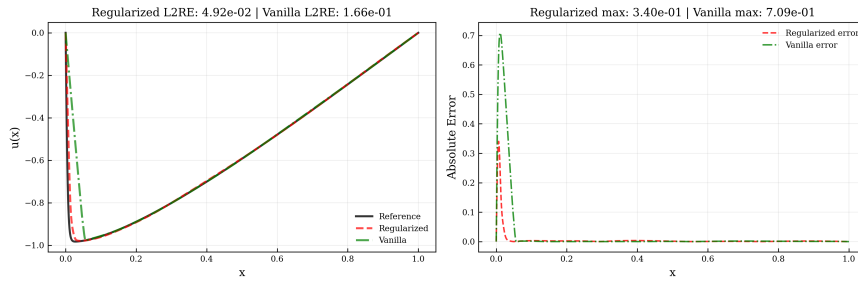


FIGURE 5.9: Comparison of FEM reference, regularized(Jacobian Regularization) PINNs, and vanilla models for the Burgers' equation at $t = 0.5$.

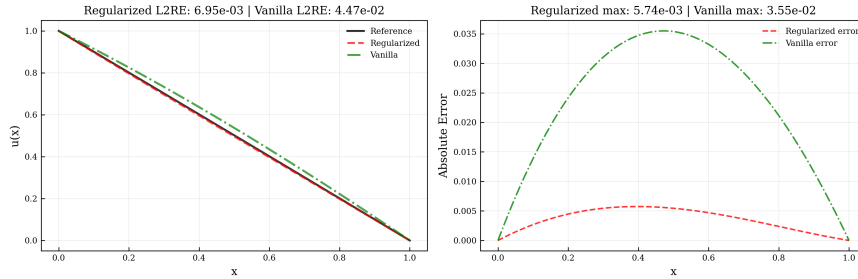


FIGURE 5.10: Comparison of FEM reference, regularized(Unimodular Regularization) PINNs, and vanilla models for the Fisher-KPP equation at $t = 0.5$.

Finally, the best-performing regularized models are compared against the vanilla PINN. For this comparison, the regularization configurations that achieved the best results in the previous experiments are selected. Specifically, these are: UR with $\lambda = 10^{-6}$ for the Fisher-KPP equation, L_1 with $\lambda = 10^{-5}$ for the Allen-Cahn equation, and JR with $\lambda = 10^{-1}$ for the Burgers equation. The comparison is presented by plotting the numerical reference solution, the regularized PINN solution, and the vanilla PINN solution on the left-hand side, while the corresponding absolute errors between the reference solution and each PINN prediction are shown on the right-hand side. This layout makes it possible to visually assess both the qualitative differences between the predicted solutions and their accuracy relative to the numerical reference. Figures 5.8, 5.9, and 5.10 illustrate these plots for each of the equations.

It is observed that the application of appropriately selected regularization techniques leads to a noticeable improvement in solution accuracy compared to the vanilla

PINN. In particular, for different benchmark equations the relative L_2 error differs by around one order of magnitude between the vanilla and regularized models, favoring the latter.

5.4 Chapter Summary

The conducted experiments confirm the central hypothesis of this work: regularized models consistently outperform the vanilla PINNs, without introducing any architectural modifications. Across all benchmark equations, regularizations have shown impact on the training dynamics and accuracy of PINNs. It was shown that carefully selected regularization can substantially improve solution quality.

A key finding of this study is that no single regularization method is universally optimal. The effectiveness of each technique is dependent on the benchmark equation. For the Fisher–KPP equation, gPINN achieved the lowest error. For the Burgers’ equation UR and JR provided the best results. For the Allen–Cahn equation, the best performance was achieved by L_1 regularization, highlighting that simple methods can improve PINNs performance significantly.

Another observation is that the optimal regularization coefficient is highly problem-dependent, with best-performing values varying significantly across equations. This suggests that hyperparameter tuning is essential, and that neither the coefficients nor the optimal regularization strategies can be transferred directly between problems. Also, regularization terms often lie at the boundaries of the explored range. It suggests that further improvements may be achievable by expanding the coefficient search space.

From the perspective of training dynamics, the results show that regularization does not fundamentally alter convergence trajectories, but rather refines them. Most methods follow similar optimization paths, with improvements emerging in later training stages. The 1-dimensional solution and heatmap analyses further reveal that regularization primarily improves performance in regions of high complexity.

While this improvement is significant, it is not trivial to achieve in practice — it requires substantial computational effort, careful hyperparameter tuning, and a robust experimental pipeline.

These results are fully reproducible and reliable, as the source code is publicly available and all experiments were conducted under specified random seeds. The findings should be considered architecture- and problem-specific: the observed improvements apply to the particular PINN architecture, PDE setup, and regularization objective used in the experiments.

Chapter 6

Conclusions

The objective of this work is to systematically investigate the impact of regularization techniques on the training behavior of PINNs. Particular attention is given to improving model performance without introducing significant architectural modifications. Specifically, this study examines how several regularization strategies — UR [20], JR [2], L_1 , L_2 , and gPINN [26] — affect convergence dynamics and solution accuracy.

A secondary objective is to establish a unified PINN-based framework for the systematic evaluation of regularization strategies. This framework enables consistent benchmarking across multiple DE and supports fully reproducible experimental results.

In addition, this work explores the use of JR within the PINN framework, which has not been previously studied in this context. It also investigates the application of L_1 and L_2 regularization to the specific objectives considered in this research.

6.1 Results Summary

A systematic comparison of the selected regularization techniques was conducted on three benchmark equations: Allen–Cahn, Fisher–KPP, and Burgers’ equations. For each benchmark equation, five independent runs were performed using fixed initial model states and controlled random seeds to ensure a fair comparison. The experimental results identified the best-performing regularization technique for each benchmark equation: gPINN for Fisher–KPP, UR and JR for Burgers’ equation, and L_1 regularization for the Allen–Cahn equation. The results show that there is no single regularization strategy that is optimal across all benchmark equations. Instead, most equations require a distinct regularization term, indicating that the choice of regularization is problem-dependent.

Across all benchmark equations, the regularized PINN with optimally selected coefficients consistently surpassed the baseline model. This confirms the hypothesis that standard PINN performance can be improved without introducing architectural modifications.

From the perspective of training dynamics, the results show that regularization does not fundamentally change the overall convergence trajectories. Rather, it refines the training process by improving model behavior at later stages of optimization. In particular, regularized models demonstrate better performance in complex solution regions compared with vanilla PINNs.

The newly introduced Jacobian Regularization demonstrated the second-best results for the complex Burgers’ benchmark equation. In addition, L_1 regularization achieved the best results for the Allen–Cahn equation, showing that even simple regularization methods can significantly improve PINN performance. To the best of our

knowledge, this is the first use of JR in the PINN field and the first application of L_1 regularization to this type of objective.

To conduct these experiments, a PINN comparison framework was developed. The framework supports flexible definitions of PDEs, ICs, BCs, computational domains, random seeds, model configurations, reference solutions, exact solutions, and other experimental components.

The main result of this work is that no single regularization strategy is universally optimal across different benchmark equations. Each benchmark equation has its own regularization strategy that yields the best results. Moreover, the experiments show that previously unexplored approaches, such as JR and the application of L_1 regularization to the considered objective, can achieve strong results compared with other regularization methods. The regularization strategies and coefficients identified in this work can serve as starting points for future hyperparameter searches.

6.2 Limitations

The results show that the optimal choice of regularization is highly dependent on the considered PDE. It is also possible that changing the boundary or initial conditions may alter the optimal regularization strategy even for the same benchmark equation. This aspect requires further investigation in future research.

6.3 Future Work

A future research direction is to investigate how these regularization techniques affect the multi-head setup. Since the developed PINN framework already supports multi-head configurations, extending the analysis to this setting represents a natural continuation of this work.

Appendix A

Appendix

A.1 Definitions

Forward problem. In the context of DEs, a forward problem consists of computing the system’s response given a fully specified mathematical model — including the governing PDE, initial conditions, boundary conditions, and all physical parameters.

Inverse problem. An inverse problem reverses this paradigm: one or more components of the model—typically unknown parameters, boundary conditions, or source terms—are inferred from (often sparse or noisy) observations of the state u . Inverse problems are generally more challenging than their forward counterparts: solutions may not be unique and can be highly sensitive to noise or perturbations in the data, necessitating regularisation strategies.

Analytical solution. A closed-form expression $u(x, t)$ is derived through exact mathematical techniques — separation of variables, Fourier series, Green’s functions, and related methods. Analytical solutions exist only for problems with simple geometry, homogeneous media, and tractable boundary conditions; however, they provide exact reference benchmarks and offer clear insight into the role of each parameter.

Numerical solution. The continuous domain is discretised — spatially via finite differences (FDM), finite elements (FEM), or finite volumes (FVM) — and time integration is performed using explicit or implicit schemes, with the resulting algebraic system solved iteratively. Numerical methods are broadly applicable to complex geometries and nonlinear problems, though convergence depends on mesh resolution and re-solving is required whenever parameters change.

Domain. In the context of PINNs, the domain refers to the region in space and/or time over which the governing physical equations are solved. It specifies the bounds within which the neural network is trained to approximate the solution.

A.2 Nonlinear Differential Equations

Differential equations play a fundamental role in scientific computing, as they are used to describe a wide range of physical phenomena, including heat conduction, fluid dynamics, and wave propagation. In general, a DE defines a relationship between an unknown function and its derivatives.

DEs can be broadly classified into two main categories:

- **Ordinary Differential Equations (ODEs)**, which involve a single independent variable. A general form of an ODE can be written as:

$$F\left(x, u, \frac{du}{dx}, \frac{d^2u}{dx^2}, \dots\right) = 0 \quad (\text{A.1})$$

- **Partial Differential Equations (PDEs)**, which involve multiple independent variables. A general form of a PDE is:

$$F\left(x, t, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial t}, \dots\right) = 0 \quad (\text{A.2})$$

A DE is called *linear* if the unknown function and all of its derivatives appear linearly; that is, they are not multiplied together nor raised to powers other than one. Consequence of linearity is the *superposition principle*: if u_1 and u_2 are solutions, then any linear combination $c_1u_1 + c_2u_2$, where c_1, c_2 are constants, is also a solution.

A DE that does not satisfy these conditions is called *nonlinear*.

A.3 Initial and boundary conditions

A DE defines a relationship between an unknown function and its derivatives, such an equation admits infinitely many solutions. To obtain a unique solution, it is necessary to specify additional constraints, known as initial and/or boundary conditions.

For ordinary differential equations (ODEs), a unique solution is determined by specifying initial conditions. Initial condition is equation state definition at initial parameters. For example, a first-order ODE can be written as:

$$\frac{du}{dx} = f(x, u), \quad u(x_0) = u_0 \quad (\text{A.3})$$

where $u(x_0) = u_0$ defines value of the equation at spatial coordinate x_0 .

For PDEs, both initial and boundary conditions are required. An example of PDE is:

$$\begin{aligned} \frac{\partial u}{\partial t} &= f\left(x, t, u, \frac{\partial u}{\partial x}, \dots\right), \quad x \in \Omega, \quad t > 0 \\ u(x, 0) &= u_0(x), \quad x \in \Omega \\ u(x, t) &= g(x, t), \quad x \in \partial\Omega \end{aligned} \quad (\text{A.4})$$

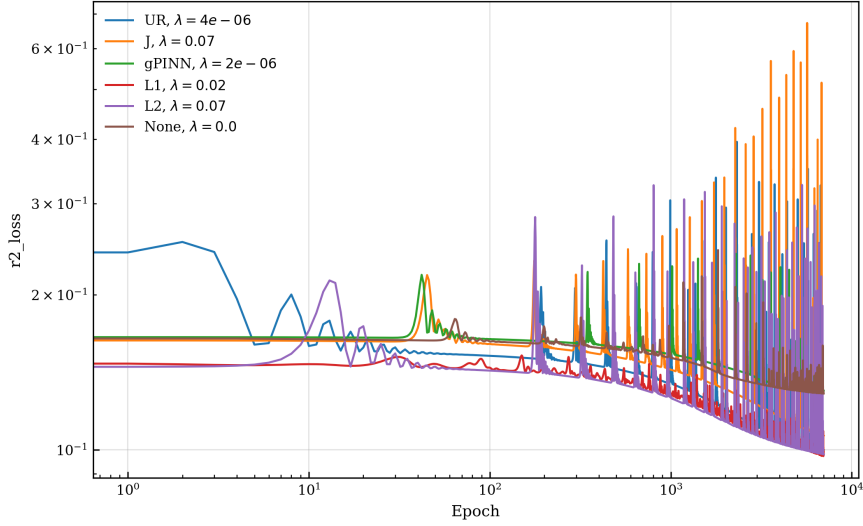
where function $g(x, t)$ defines state at boundary conditions, and $\partial\Omega$ is the domain of the equation boundaries.

A.4 Exponential Moving Average

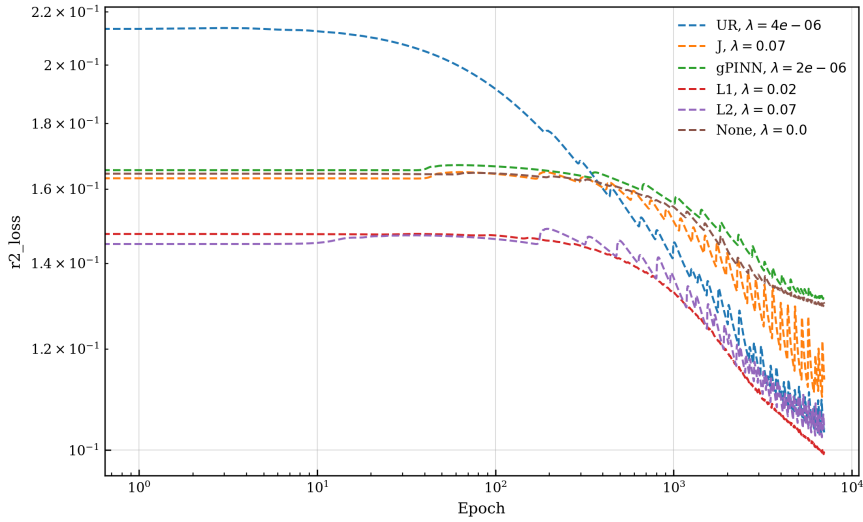
EMA is a smoothing technique that computes a weighted average of a sequence, assigning exponentially decreasing weights to older values. Given a sequence x_t , the EMA is defined as:

$$\hat{x}_t = \alpha \hat{x}_{t-1} + (1 - \alpha)x_t \quad (\text{A.5})$$

where $\alpha \in (0, 1)$ is the decay factor controlling the balance between historical and current values. A lower α produces smoother estimates by emphasizing past observations, while a higher α makes the estimate more responsive to recent changes.



(A) Default plot



(B) Plot with EMA, $\alpha = 0.005$

FIGURE A.1: Example of Exponential Moving Average method applied to Fisher-KPP equation training data.

A.5 Hyperparameters configuration

Hyperparameters used in the experiments are provided in Table A.1.

TABLE A.1: Hyperparameter configuration for each benchmark equation. E_1 denotes experiment 5.1, while E_2 , E_3 - experiments 5.2 and 5.3

Hyperparameter	Allen-Cahn	Burgers'	Fisher-KPP
Spatial domain x	$[-1, 1]$	$[0, 1]$	$[0, 1]$
Time domain t	$[0, 1]$	$[0, 1]$	$[0, 1]$
Grid size (N_t, N_x)	$(32, 64)$	$(64, 64)$	$(64, 64)$
Equation parameter	$\varepsilon = 10^{-4}$	$\nu = 3.18 \times 10^{-3}$	$r = 10^{-2}$
Body architecture	$(20, 20, 20, 20, 20, 20)$	$(64, 64, 64, 64)$	$(20, 20, 20, 20)$
Head architecture	$(16, 16)$	$(32, 32)$	$(16, 16)$
Latent basis length	8	32	8
Bias	True	True	False
Optimizer	Adam + L-BFGS	Adam + L-BFGS	Adam + L-BFGS
Adam learning rate	10^{-3}	10^{-3}	10^{-3}
Adam epochs	5000	5000	5000
L-BFGS epochs (E_1)	150	150	150
L-BFGS epochs (E_2, E_3)	300	300	300
L-BFGS learning rate	0.5	0.5	0.5
Evaluation time t_i	0.5	0.5	0.5
Evaluation step Δx	10^{-4}	10^{-4}	10^{-4}

Appendix B

Appendix

B.1 Computational Infrastructure

All experiments were conducted on a high-performance computing (HPC) cluster operating within a GPU-enabled environment, with job scheduling managed through the Slurm workload manager. Each experimental run was configured as a single-task job allocated 4 CPU cores and 64 GB of system memory, with a maximum wall-clock time of 5 days to accommodate the computational demands of long-running training procedures.

The full experimental pipeline comprised 261 individual runs, structured as follows: 153 runs (3×51) for hyperparameter search over regularization terms; 90 runs ($3 \times 5 \times 6$) for estimation of mean and standard deviation across multiple random seeds under the selected regularization configurations; and 18 runs (3×6) for the final evaluation of regularization performance. Collectively, these 261 runs amounted to a total compute time of approximately 104 GPU-hours. It should be noted that this figure excludes intermediate exploratory runs conducted during the development phase, which also contributed to the total computational budget. The hardware specification of the cluster node used throughout is summarized in Table B.1.

TABLE B.1: HPC Cluster Hardware Specification

Component	Specification
GPU	NVIDIA A40
CPU	AMD EPYC 7763
CPU cores per task	4
System memory	64 GB RAM

Prior to deployment on the HPC cluster, the codebase was developed, tested, and iteratively refined in Google Colaboratory, which provided an accessible interactive environment for rapid prototyping. The Colaboratory runtime utilized the following hardware configuration (Table B.2).

TABLE B.2: Google Colaboratory Hardware Specification

Component	Specification
GPU	NVIDIA T4 (16 GB VRAM)
CPU	AMD EPYC 7B12
System memory	12 GB RAM

Bibliography

- [1] Zhongkai Hao et al. “Pinnacle: A comprehensive benchmark of physics-informed neural networks for solving pdes”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 76721–76774.
- [2] Judy Hoffman, Daniel A Roberts, and Sho Yaida. “Robust learning with jacobian regularization”. In: *arXiv preprint arXiv:1908.02729* (2019).
- [3] Davyd Ilnytskyi. *Regularization methods in PINNs for Nonlinear Differential Equations*. <https://github.com/DavidIlnytskyi/Regularization-methods-in-PINNs-for-Nonlinear-Differential-Equations>. Accessed: 2026-04-29. 2026.
- [4] Ameya D Jagtap, Ehsan Kharazmi, and George Em Karniadakis. “Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems”. In: *Computer Methods in Applied Mechanics and Engineering* 365 (2020), p. 113028.
- [5] George Em Karniadakis et al. “Physics-informed machine learning”. In: *Nature Reviews Physics* 3.6 (2021), pp. 422–440.
- [6] Siavash Khodakarami et al. *Spectral bias in physics-informed and operator learning: Analysis and mitigation guidelines*. 2026. arXiv: [2602.19265](https://arxiv.org/abs/2602.19265) [cs.LG]. URL: <https://arxiv.org/abs/2602.19265>.
- [7] Diederik Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations* (Dec. 2014).
- [8] Elham Kiyani et al. “Which optimizer works best for physics-informed neural networks and Kolmogorov-Arnold networks?” In: *arXiv e-prints* (2025), arXiv–2501.
- [9] Isaac Lagaris, Aristidis Likas, and Dimitrios Fotiadis. “Artificial neural networks for solving ordinary and partial differential equations”. In: *IEEE Transactions on Neural Networks* 9 (Sept. 1998), pp. 987–1000. DOI: [10.1109/72.712178](https://doi.org/10.1109/72.712178).
- [10] Dong C Liu and Jorge Nocedal. “On the limited memory BFGS method for large scale optimization”. In: *Mathematical programming* 45.1 (1989), pp. 503–528.
- [11] Wing Kam Liu, Shaofan Li, and Harold S Park. “Eighty years of the finite element method: birth, evolution, and future”. In: *Archives of Computational Methods in Engineering* 29.6 (2022), pp. 4431–4453.
- [12] Lu Lu et al. “Physics-informed neural networks with hard constraints for inverse design”. In: *SIAM Journal on Scientific Computing* 43.6 (2021), B1105–B1132.
- [13] Kuang Luo et al. “Physics-informed neural networks for PDE problems: a comprehensive review”. In: *Artificial Intelligence Review* 58 (July 2025). DOI: [10.1007/s10462-025-11322-7](https://doi.org/10.1007/s10462-025-11322-7).
- [14] Paweł Maczuga and Maciej Paszyński. “Influence of activation functions on the convergence of physics-informed neural networks for 1d wave equation”. In: *International Conference on Computational Science*. Springer. 2023, pp. 74–88.

- [15] NeuroDiffEq. *Satisfying the Initial/Boundary Conditions*. 2019. URL: <https://neurodiffEq.readthedocs.io/en/latest/how.html#Satisfying-the-Initial/Boundary-Conditions>.
- [16] M. Raissi, P. Perdikaris, and G.E. Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: <https://doi.org/10.1016/j.jcp.2018.10.045>. URL: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [17] Pratik Rathore et al. “Challenges in training pinns: A loss landscape perspective”. In: *arXiv preprint arXiv:2402.01868* (2024).
- [18] Khemraj Shukla et al. “A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks”. In: *Computer Methods in Applied Mechanics and Engineering* 431 (2024), p. 117290.
- [19] Yanjie Song et al. “Loss-attentional physics-informed neural networks”. In: *Journal of Computational Physics* 501 (2024), p. 112781.
- [20] Pedro Tarancón-Álvarez et al. “Efficient PINNs via multi-head unimodular regularization of the solutions space”. In: *Communications Physics* 8.1 (2025), p. 335.
- [21] *Unimodular regularization*. 2023. URL: https://github.com/pedrota2000/Unimodular_regularization.
- [22] Sifan Wang, Yujun Teng, and Paris Perdikaris. “Understanding and mitigating gradient flow pathologies in physics-informed neural networks”. In: *SIAM Journal on Scientific Computing* 43.5 (2021), A3055–A3081.
- [23] Sifan Wang, Hanwen Wang, and Paris Perdikaris. “On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks”. In: *Computer Methods in Applied Mechanics and Engineering* 384 (2021), p. 113938.
- [24] Chenxi Wu et al. “A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks”. In: *Computer Methods in Applied Mechanics and Engineering* 403 (2023), p. 115671.
- [25] Haixu Wu et al. “Transolver: A fast transformer solver for pdes on general geometries”. In: *arXiv preprint arXiv:2402.02366* (2024).
- [26] Jeremy Yu et al. “Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems”. In: *Computer Methods in Applied Mechanics and Engineering* 393 (2022), p. 114823.
- [27] Jeremy Yu et al. “Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems”. In: *Computer Methods in Applied Mechanics and Engineering* 393 (2022), p. 114823. ISSN: 0045-7825. DOI: <https://doi.org/10.1016/j.cma.2022.114823>. URL: <https://www.sciencedirect.com/science/article/pii/S0045782522001438>.
- [28] Zhiyuan Zhao, Xueying Ding, and B Aditya Prakash. “Pinnsformer: A transformer-based framework for physics-informed neural networks”. In: *arXiv preprint arXiv:2307.11833* (2023).